

Universidad Nacional
de Educación a Distancia

Escuela Técnica Superior de
Ingeniería Informática



Universidad
Complutense de Madrid

Facultad de
Informática

Diseño de una aplicación mediante visión artificial para el desarrollo de un asistente accesible

Fco. Javier López López

Director/a: Dra. María Guijarro Mata-García
Co-director/a: Dr. Joaquín Recas Piorno

Trabajo de Fin de Máster

Máster Universitario
en Ingeniería de Sistemas y de Control

Noviembre 2026

Agradecimientos

Quisiera aprovechar estas líneas para dar las gracias a mis tutores del proyecto, a todo el profesorado, a la Universidad Nacional de Educación a Distancia (UNED), a mi familia y amigos.

Resumen

Las personas con discapacidad visual encuentran dificultades para acceder de forma autónoma a la información textual presente en el etiquetado de productos de uso cotidiano. Este Trabajo Fin de Máster presenta el diseño, desarrollo y evaluación de *Praeco*, un asistente accesible basado en visión artificial, reconocimiento óptico de caracteres y síntesis de voz, implementado para dispositivos móviles Android.

El sistema integra, en un dispositivo móvil, una cadena de procesamiento que comprende la adquisición de imágenes mediante la cámara, la evaluación automática de su calidad a partir de parámetros de nitidez, brillo y contraste, el recorte de la región de interés, el reconocimiento óptico de caracteres mediante Google ML Kit, el filtrado y reconstrucción del texto reconocido, la corrección léxica, la detección de términos asociados a alérgenos y la presentación del resultado mediante síntesis de voz.

Como evolución posterior de la adquisición se ha implementado una captura automática de una sola fotografía, con activación estable de la linterna cuando el brillo es insuficiente y mejora de imagen condicional mediante CLAHE. Estas modificaciones se describen separadamente de la validación principal del corrector.

Para mejorar los errores producidos por el reconocimiento OCR se ha desarrollado un corrector conservador que combina distancia de Levenshtein, similitud fonética, criterios de longitud, mecanismos de protección de palabras y recursos léxicos. El sistema incorpora además un mecanismo de aprendizaje adaptativo que permite obtener conocimiento a partir de los errores observados durante la fase de desarrollo. Una vez finalizado el aprendizaje, los recursos obtenidos se congelan para realizar la validación sin modificar el comportamiento del corrector durante los ensayos.

La arquitectura final se ha organizado en tres aplicaciones complementarias: *Praeco Desarrollo*, utilizada para el desarrollo, aprendizaje y evaluación experimental; *Praeco Usuario*, destinada al funcionamiento normal del asistente con los recursos de aprendizaje previamente obtenidos; y *Praeco Análisis*, desarrollada para el tratamiento y análisis de los resultados experimentales.

La evaluación final se realizó sobre 63 registros con el aprendizaje desactivado. Los resultados muestran una reducción del Character Error Rate (CER) medio de 0,0646 a 0,0621 y del Word Error Rate (WER) medio de 0,1834 a 0,1717 tras aplicar el corrector. La corrección mejoró 15 casos, empeoró 6 y mantuvo sin cambios 42, preservando además los cinco casos en los que el OCR inicial era perfecto. Estos resultados muestran una mejora global moderada y evidencian el carácter conservador del sistema de corrección, aunque también ponen de manifiesto que su efecto depende de las características del error producido por el OCR.

El prototipo desarrollado demuestra la viabilidad de integrar técnicas de visión artificial y procesamiento de texto en dispositivos móviles comerciales para facilitar el acceso autónomo a información textual. Praeco constituye así una solución de bajo coste y una plataforma

experimental sobre la que continuar desarrollando mecanismos de reconocimiento, corrección y asistencia accesible.

Palabras clave: discapacidad visual, visión artificial, reconocimiento óptico de caracteres, OCR, Google ML Kit, corrección léxica, aprendizaje adaptativo, accesibilidad, síntesis de voz, Android.

Abstract

People with visual impairments experience difficulties in independently accessing the textual information provided on the labels of everyday products. This Master's Thesis presents the design, development, and evaluation of *Praeco*, an accessible assistant based on computer vision, optical character recognition, and speech synthesis, implemented for Android mobile devices.

The system integrates, on a mobile device, a processing pipeline comprising image acquisition using the device's camera, automatic image quality assessment based on sharpness, brightness, and contrast parameters, region-of-interest cropping, optical character recognition using Google ML Kit, filtering and reconstruction of the recognized text, lexical correction, detection of terms associated with allergens, and presentation of the results through speech synthesis.

A subsequent acquisition optimisation introduces single-image automatic capture, stable auxiliary lighting under insufficient illumination, and conditional image enhancement using CLAHE. These changes are reported separately from the main validation of the lexical corrector.

To improve errors produced by the OCR process, a conservative correction system has been developed that combines Levenshtein distance, phonetic similarity, word-length criteria, word-protection mechanisms, and lexical resources. The system also incorporates an adaptive learning mechanism that allows knowledge to be obtained from errors observed during the development phase. Once the learning process has been completed, the resulting resources are frozen so that validation can be performed without modifying the behaviour of the correction system during the experiments.

The final architecture has been organised into three complementary applications: *Praeco Development*, used for development, learning, and experimental evaluation; *Praeco User*, intended for the normal operation of the assistant using the previously obtained learning resources; and *Praeco Analysis*, developed for processing and analysing the experimental results.

The final evaluation was performed on 63 records with learning disabled. The results show a reduction in the mean Character Error Rate (CER) from 0.0646 to 0.0621 and in the mean Word Error Rate (WER) from 0.1834 to 0.1717 after applying the correction system. The correction improved 15 cases, worsened 6, and left 42 unchanged, while also preserving the five cases in which the initial OCR output was error-free. These results show a moderate overall improvement and demonstrate the conservative nature of the correction system, while also indicating that its effect depends on the characteristics of the errors produced by the OCR process.

The developed prototype demonstrates the feasibility of integrating computer vision and text-processing techniques into commercially available mobile devices to facilitate indepen-

dent access to textual information. Praeco therefore constitutes a low-cost solution and an experimental platform for the further development of recognition, correction, and accessible assistance mechanisms.

Keywords: visual impairment, computer vision, optical character recognition, OCR, Google ML Kit, lexical correction, adaptive learning, accessibility, speech synthesis, Android.

Contents

1. Introducción **1**
1.1. Motivación 1
1.2. Propuesta y objetivos 2

2. Estado del Arte **5**
2.1. Soluciones de asistencia visual 6
2.2. Tecnologías relevantes para Praeco 6
2.2.1. Reconocimiento óptico de caracteres 6
2.2.2. Corrección léxica posterior al OCR 6
2.2.3. Procesamiento local y privacidad 7
2.3. Praeco y su aportación 7
2.4. Brecha abordada por el trabajo 8

3. Materiales y métodos **9**
3.1. Arquitectura de la aplicación y funcionamiento detallado de Praeco 9
3.1.1. Flujo general de procesamiento 10
3.1.2. Adquisición de la imagen y región de interés 11
3.1.3. Evaluación de la calidad de imagen 11
3.1.4. Control automático de iluminación y estabilización 12
3.1.5. Preprocesamiento condicional mediante CLAHE 12
3.1.6. Reconocimiento OCR mediante Google ML Kit 13
3.1.7. Filtrado del OCR 13
3.1.8. Reconstrucción y protección de tokens 13
3.1.9. Corrector léxico híbrido 14
3.1.10. Distancia de Levenshtein 14
3.1.11. Información fonética 15
3.1.12. Diccionario adaptativo y aprendizaje incremental 15
3.1.13. Corrección contextual mediante n-gramas 15
3.1.14. Detección de alérgenos 16
3.1.15. Síntesis de voz 17
3.1.16. Evaluación mediante ground truth, CER y WER 17

3.1.17. Medición temporal . . . 18
3.1.18. Almacenamiento experimental . . . 18
3.1.19. Praeco Análisis . . . 19
3.1.20. Separación entre Praeco Desarrollo y Praeco Usuario . . . 20
3.2. Tecnologías y Dependencias Utilizadas . . . 20
3.3. Justificación de la elección de Flutter y Dart . . . 21
3.4. Ventajas específicas de Flutter en este proyecto . . . 22
3.5. Conclusión tecnológica . . . 23

4. Resultados **25**
4.1. Introducción . . . 25
4.2. Métricas de evaluación . . . 26
4.2.1. Character Error Rate (CER) . . . 26
4.2.2. Word Error Rate (WER) . . . 26
4.2.3. Criterio de comparación . . . 26
4.3. Resultados preliminares . . . 28
4.4. Evolución del corrector . . . 28
4.5. Ensayos con aprendizaje activo . . . 29
4.6. Validación con aprendizaje desactivado . . . 30
4.7. Importancia de la auditoría del *ground truth* . . . 31
4.8. Interpretación conjunta de los modos de aprendizaje . . . 31
4.9. Análisis de las causas de error . . . 31
4.10. Análisis temporal . . . 32
4.11. Papel de Praeco Análisis . . . 33
4.12. Discusión de los resultados . . . 37
4.13. Ensayos exploratorios de captura rápida e iluminación asistida . . . 39
4.14. Conclusiones del análisis experimental . . . 40

5. Conclusiones y trabajos futuros **43**
5.1. Conclusiones . . . 43
5.2. Aportaciones principales del trabajo . . . 45
5.3. Trabajos futuros . . . 46

Bibliografía y referencias **48**

A. Software desarrollado **51**
A.1. Introducción . . . 51
A.2. Repositorio y versión de referencia . . . 51
A.3. Estructura del repositorio . . . 51
A.4. Praeco Desarrollo . . . 52

A.5. Praeco Usuario 52
A.6. Praeco Análisis 52
A.7. Requisitos de compilación 52
A.8. Reproducibilidad 52
A.9. Datos generados durante la experimentación 53
A.10.Disponibilidad del software 53

List of Figures

3.1. Arquitectura general del ecosistema Praeco. 9
3.2. Flujo de captura rápida y preprocesamiento condicional de Praeco. La captura se realiza al cumplir los criterios de calidad; la linterna permanece encendida hasta finalizarla cuando es necesaria. 10
3.3. Evaluación de calidad de la imagen. 12
3.4. Proceso de decisión del corrector léxico de Praeco. 14
3.5. Esquema del aprendizaje incremental de Praeco Desarrollo. 16
3.6. Relación entre Praeco Desarrollo y Praeco Análisis. 19
3.7. Separación funcional entre Praeco Desarrollo y Praeco Usuario. 20

4.1. Flujo de evaluación experimental de Praeco. 27
4.2. Pantallas de ejemplo de Praeco Análisis. 36

Índice de tablas

2.1. Comparación cualitativa entre soluciones de asistencia visual y Praeco. . . . 8

4.1. Resultados de los cinco experimentos preliminares. 28
4.2. Resumen de un ensayo con aprendizaje activo. 29
4.3. Evaluación intermedia con aprendizaje desactivado y *ground truth* auditado. 30
4.4. Tiempos medios de procesamiento por producto durante la fase de validación. 33
4.5. Dos ensayos exploratorios de captura rápida con umbral de iluminación 150. 39

Chapter 1

Introducción

1.1. Motivación

En la última década, la proliferación de los dispositivos móviles y el avance de la inteligencia artificial han transformado radicalmente la interacción entre los seres humanos y su entorno. Sin embargo, a pesar de estos avances, las personas con discapacidad visual siguen enfrentando desafíos significativos en tareas cotidianas que el resto de la población da por sentadas. La lectura de indicaciones en la vía pública, la interpretación de etiquetas informativas o el acceso a documentos impresos en tiempo real son barreras que limitan la autonomía personal y la inclusión social.

Este proyecto se centra en el desarrollo de una aplicación móvil multiplataforma que actúa como un puente sensorial. Utilizando técnicas de visión artificial y procesamiento de lenguaje natural, el sistema busca convertir información visual en estímulos auditivos, permitiendo que el usuario "lea" su entorno de manera fluida y eficiente.

La motivación principal de este trabajo surge de la brecha existente entre las capacidades tecnológicas actuales y su aplicación real en el ámbito de la accesibilidad. Aunque existen herramientas comerciales de asistencia, muchas presentan barreras de coste, requieren hardware específico o carecen de una interfaz adaptada a la baja visión.

La elección de Flutter como entorno de desarrollo y la integración de servicios de OCR de última generación responde a la necesidad de crear una herramienta democrática: una solución de software que pueda ejecutarse en teléfonos inteligentes convencionales sin necesidad de inversiones adicionales. La posibilidad de mejorar la calidad de vida de los usuarios mediante la ingeniería de software, transformando un dispositivo común en un asistente inteligente y proactivo, constituye el motor fundamental de esta investigación.

Para identificar visualmente la aplicación he creado el siguiente icono al que he llamado Praeco.



PRAECO

Los praecones (singular: ***praeco****) eran los heraldos oficiales en la Antigua Roma. Su función era ser la "voz pública" del Estado.*

1.2. Propuesta y objetivos

El objetivo general de este Trabajo Fin de Máster es diseñar, desarrollar y evaluar un sistema móvil de asistencia para personas con discapacidad visual o baja visión, denominado *Praeco*, capaz de reconocer información textual presente en el etiquetado de productos, mejorar de forma conservadora los errores producidos durante el reconocimiento OCR, detectar posibles alérgenos y comunicar la información obtenida mediante síntesis de voz.

Como resultado de la evolución del proyecto, Praeco se ha estructurado en un ecosistema formado por tres aplicaciones independientes pero relacionadas entre sí: **Praeco Desarrollo**, **Praeco Usuario** y **Praeco Análisis**. Esta separación permite diferenciar claramente las funciones de desarrollo, aprendizaje y evaluación experimental de aquellas necesarias para el funcionamiento de la aplicación destinada al usuario final.

Los objetivos específicos del proyecto son los siguientes:

- Diseñar e implementar una arquitectura de procesamiento capaz de adquirir imágenes mediante la cámara del dispositivo móvil, seleccionar una región de interés y evaluar automáticamente su calidad mediante parámetros de nitidez, brillo y contraste.

- Estudiar la optimización de la captura automática mediante análisis previo de fotogramas, iluminación auxiliar y mejora condicional del contraste con CLAHE.

- Integrar un motor de reconocimiento óptico de caracteres basado en Google ML Kit que permita obtener el texto presente en etiquetas y otros soportes impresos utilizando un dispositivo Android convencional.

- Desarrollar un sistema híbrido y conservador de corrección de errores OCR basado en distancia de Levenshtein, semejanza fonética, diccionarios lingüísticos, reglas de protección, vocabulario adaptativo y mecanismos de aprendizaje incremental.

- Implementar mecanismos de filtrado y reconstrucción del texto reconocido que permitan reducir información irrelevante y preparar los tokens para las etapas posteriores de corrección y análisis.

- Incorporar un módulo de detección de alérgenos capaz de identificar términos asociados a gluten, lácteos, huevo, frutos secos, cacahuete, soja, pescado y marisco, considerando además determinadas expresiones de ausencia, como *«sin gluten»*, con el objetivo de reducir falsos positivos.

- Proporcionar una salida accesible mediante síntesis de voz (*Text-to-Speech*, TTS), de forma que el texto reconocido, corregido y las advertencias generadas puedan ser comunicados auditivamente al usuario.

- Desarrollar **Praeco Desarrollo** como aplicación destinada a la experimentación y evolución del sistema. Esta versión incorpora las funciones de captura, OCR, corrección, detección de alérgenos y TTS, junto con mecanismos específicos de introducción del *ground truth*, aprendizaje adaptativo, trazabilidad, almacenamiento de experimentos y evaluación cuantitativa mediante CER y WER.

- Implementar en Praeco Desarrollo un sistema de aprendizaje incremental capaz de registrar asociaciones entre errores OCR y palabras correctas, controlar su frecuencia y posibles conflictos, y generar progresivamente un diccionario adaptativo con vocabulario válido aprendido durante la fase experimental.

- Desarrollar **Praeco Usuario** como una aplicación independiente destinada al uso normal del asistente. Esta versión utiliza los recursos lingüísticos previamente aprendidos y congelados, conservando únicamente las funciones necesarias para el usuario: cámara, evaluación de calidad, OCR, filtrado, corrección, detección de alérgenos y síntesis de voz, sin incorporar *ground truth*, aprendizaje ni herramientas de evaluación experimental.

- Desarrollar **Praeco Análisis** como una tercera aplicación independiente destinada al tratamiento de los resultados experimentales generados por Praeco Desarrollo. Esta aplicación permite seleccionar el fichero de experimentos y realizar el análisis estadístico de las métricas CER y WER, la mejora producida por el corrector, los tiempos de procesamiento, las variables de calidad de imagen, los casos de mejora y empeoramiento y la separación entre experimentos de aprendizaje y validación.

- Evaluar objetivamente el funcionamiento del sistema mediante las métricas *Character Error Rate* (CER) y *Word Error Rate* (WER), comparando el texto producido originalmente por el OCR con el obtenido después de aplicar el corrector de Praeco.

- Analizar el coste computacional de las principales etapas del procesamiento mediante el registro de los tiempos de captura, reconocimiento OCR, corrección y detección de alérgenos, permitiendo estudiar la viabilidad del sistema para su utilización interactiva en un dispositivo móvil.

- Establecer una metodología experimental que mantenga separados los procesos de aprendizaje y validación. Una vez estabilizados el código y los recursos lingüísticos, estos se congelan para realizar una validación independiente, evitando que los datos empleados durante el aprendizaje condicionen los resultados finales.
- Diseñar una interfaz sencilla y accesible, orientada a reducir la dependencia de elementos exclusivamente visuales y favorecer la interacción mediante información auditiva.

La arquitectura resultante establece, por tanto, una separación funcional entre las tres aplicaciones. **Praeco Desarrollo** permite construir, aprender y evaluar el sistema; **Praeco Usuario** constituye el asistente operativo destinado al usuario final utilizando recursos previamente validados; y **Praeco Análisis** proporciona las herramientas necesarias para estudiar estadísticamente los resultados experimentales. Esta división facilita la trazabilidad del proceso de desarrollo y permite realizar una evaluación metodológicamente independiente del sistema antes de transferir los recursos validados a la aplicación de usuario.

Chapter 2

Estado del Arte

La asistencia tecnológica destinada a personas con discapacidad visual ha evolucionado desde dispositivos específicos hacia soluciones basadas en teléfonos móviles, visión artificial y procesamiento local o remoto de la información. El objetivo común de estas tecnologías es aumentar la autonomía del usuario ante tareas cotidianas como la lectura de documentos, la identificación de productos o el acceso a información textual. La literatura sobre tecnología asistencial destaca la importancia de combinar funcionalidad, accesibilidad y facilidad de uso en este tipo de sistemas Hersh and Johnson [2008].

En el ámbito del procesamiento de imágenes, las técnicas de visión artificial permiten adquirir, mejorar y analizar imágenes capturadas en condiciones reales Gonzalez and Woods [2018], Szeliski [2011]. Sobre esta base, el reconocimiento óptico de caracteres (OCR) transforma el contenido visual en texto susceptible de ser procesado posteriormente. Los motores OCR modernos ofrecen resultados útiles en numerosos escenarios, aunque su precisión continúa dependiendo de factores como el enfoque, la iluminación, el contraste, la perspectiva, el tamaño de los caracteres y la disposición del texto Smith [2007].

La corrección posterior al OCR constituye, por tanto, una etapa relevante cuando se trabaja con imágenes no controladas. Entre las técnicas clásicas se encuentra la distancia de Levenshtein, que cuantifica el número mínimo de inserciones, eliminaciones y sustituciones necesarias para transformar una cadena en otra Levenshtein [1966]. Esta medida puede combinarse con información fonética, reglas lingüísticas y modelos de contexto. Los modelos basados en n-gramas permiten representar regularidades locales del lenguaje y constituyen una herramienta habitual dentro del procesamiento del lenguaje natural Jurafsky and Martin [2023].

Para evaluar cuantitativamente un sistema OCR resulta necesario comparar el texto reconocido con una referencia o *ground truth*. En este trabajo se emplean principalmente el *Character Error Rate* (CER) y el *Word Error Rate* (WER), métricas derivadas de las operaciones de edición necesarias para transformar la hipótesis reconocida en la referencia. Junto con estas métricas de precisión, el tiempo de procesamiento resulta relevante para valorar la viabilidad de una solución interactiva.

2.1. Soluciones de asistencia visual

Existen aplicaciones comerciales que demuestran la utilidad de la visión artificial y el OCR como tecnologías de apoyo. *Seeing AI*, desarrollada por Microsoft, integra funciones destinadas a interpretar información visual y textual para usuarios con discapacidad visual Microsoft [2026]. Por su parte, *Google Lens* combina reconocimiento visual y procesamiento de texto dentro del ecosistema de Google Google [2026a]. Estas soluciones muestran el potencial de los dispositivos móviles como plataforma de asistencia, aunque su propósito es general y no coincide necesariamente con el flujo específico de captura, corrección léxica, evaluación experimental y detección de términos asociados a alérgenos desarrollado en Praeco.

Desde el punto de vista de la plataforma, Android proporciona acceso a cámara, almacenamiento y servicios de accesibilidad Google [2026b]. Flutter y Dart permiten construir una aplicación móvil multiplataforma a partir de una base de código común Google [2026c,d]. En Praeco se ha empleado Google ML Kit para realizar el reconocimiento óptico de caracteres en el dispositivo Google [2026e], junto con síntesis de voz para comunicar los resultados al usuario.

2.2. Tecnologías relevantes para Praeco

2.2.1. Reconocimiento óptico de caracteres

El OCR constituye el núcleo de la extracción textual. En un escenario de etiquetado de productos, la entrada no procede de documentos digitalizados en condiciones controladas, sino de fotografías realizadas con un teléfono móvil. Por ello, la calidad de la captura condiciona directamente el texto obtenido. Praeco incorpora una etapa previa de evaluación de nitidez, brillo y contraste para reducir la probabilidad de procesar imágenes claramente inadecuadas.

2.2.2. Corrección léxica posterior al OCR

La salida del OCR puede contener sustituciones, inserciones, eliminaciones o segmentaciones incorrectas. Praeco aborda este problema mediante un corrector conservador que combina distancia de Levenshtein Levenshtein [1966], similitud fonética, criterios dependientes de la longitud, reglas de protección y recursos léxicos. El objetivo no es modificar sistemáticamente el texto reconocido, sino intervenir únicamente cuando existe evidencia suficiente para proponer una alternativa más fiable.

El proyecto contempla asimismo recursos de contexto basados en n-gramas. No obstante, aunque dichos recursos se encuentran disponibles en la implementación, la corrección contextual mediante n-gramas no forma parte de la ruta activa utilizada en la evaluación final.

Esta distinción es importante para no atribuir a ese mecanismo las mejoras cuantitativas presentadas posteriormente.

2.2.3. Procesamiento local y privacidad

La ejecución en el propio dispositivo reduce la necesidad de transmitir imágenes de etiquetas a servicios externos y disminuye la dependencia de conectividad. Esta característica resulta especialmente relevante en una aplicación asistencial que puede utilizarse en contextos cotidianos y con información capturada directamente por la cámara. La arquitectura final de Praeco está orientada a Android y utiliza componentes locales para las etapas principales de captura, OCR, corrección, detección y síntesis de voz.

2.3. Praeco y su aportación

Praeco se plantea como un prototipo experimental y asistencial centrado en el acceso a información textual presente en etiquetas. A diferencia de una herramienta OCR genérica, integra en un mismo flujo la adquisición de la imagen, su evaluación de calidad, la selección de una región de interés, el reconocimiento mediante ML Kit, el filtrado y reconstrucción del texto, la corrección léxica, la detección de términos asociados a alérgenos y la síntesis de voz.

El corrector protege números, cantidades, palabras cortas, términos previamente validados y otras expresiones que no deben modificarse sin evidencia suficiente. Durante la fase de desarrollo se incorpora además aprendizaje adaptativo a partir de errores observados y validados. Para la evaluación final, estos recursos se congelan, evitando que las muestras de validación modifiquen simultáneamente el conocimiento utilizado por el corrector.

La Tabla 2.1 resume, de forma cualitativa, algunas diferencias entre soluciones de propósito general y el enfoque específico adoptado en Praeco. La comparación pretende contextualizar funcionalidades y no establecer una clasificación de precisión, ya que no se ha realizado un ensayo experimental común entre las aplicaciones.

La comparación pone de manifiesto que la aportación principal de Praeco no consiste en competir con plataformas comerciales de visión artificial de propósito general, sino en estudiar de forma reproducible una cadena concreta de procesamiento y evaluar cuantitativamente el efecto de la corrección posterior al OCR. La arquitectura separa además el entorno de desarrollo y aprendizaje, la aplicación destinada al usuario y la herramienta de análisis experimental.

Table 2.1: Comparación cualitativa entre soluciones de asistencia visual y Praeco.

Sistema	**OCR**	**Orientación**	**Corrección propia evaluada**	**Alérgenos**
Seeing AI Microsoft [2026]	Sí	Asistencia visual general	No evaluada en este TFM	No específica
Google Lens Google [2026a]	Sí	Búsqueda y análisis visual general	No evaluada en este TFM	No específica
Praeco	**Sí**	**Etiquetado y accesibilidad**	**Sí**	**Detección de términos**

2.4. Brecha abordada por el trabajo

A partir del análisis anterior, el presente TFM aborda una combinación de necesidades: ejecución sobre un dispositivo móvil comercial, control explícito de la calidad de captura, corrección léxica conservadora, aprendizaje adaptativo durante el desarrollo, detección de términos asociados a alérgenos, salida accesible mediante voz y evaluación cuantitativa mediante CER, WER y tiempos de procesamiento.

El interés del trabajo reside, por tanto, tanto en el prototipo funcional como en la metodología experimental utilizada para estudiar sus limitaciones. Los capítulos siguientes describen la arquitectura de Praeco, los mecanismos implementados y la evaluación realizada con los recursos de aprendizaje congelados.

Chapter 3

Materiales y métodos

3.1. Arquitectura de la aplicación y funcionamiento detallado de Praeco

Praeco es un asistente accesible para dispositivos Android orientado a personas con discapacidad visual o baja visión. Su objetivo es utilizar la cámara del teléfono y técnicas de visión artificial para reconocer el texto presente en el etiquetado de productos, corregir de forma conservadora determinados errores producidos durante el reconocimiento OCR, identificar posibles alérgenos y comunicar el resultado mediante síntesis de voz.

En la arquitectura actual se distinguen tres aplicaciones: **Praeco Desarrollo**, utilizada para la experimentación, aprendizaje, trazabilidad y evaluación; **Praeco Usuario**, que constituye la aplicación operativa final con los recursos aprendidos congelados; y **Praeco Análisis**, destinada al tratamiento estadístico de los resultados almacenados por la versión de Desarrollo.

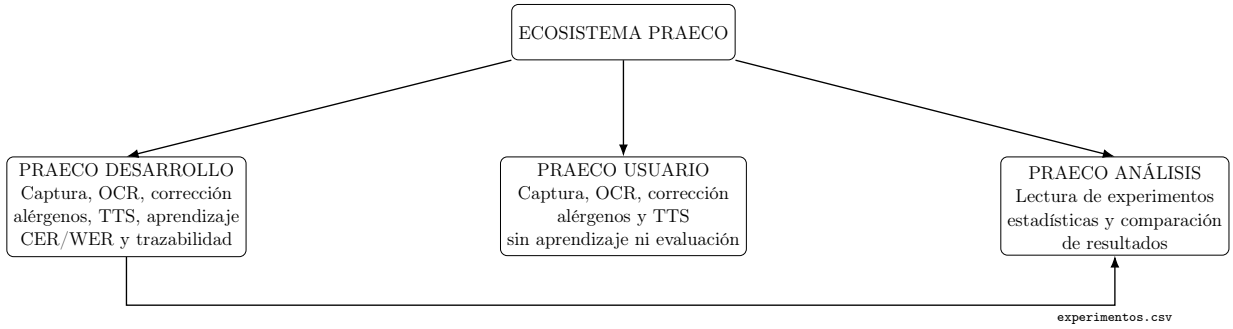


Figure 3.1: Arquitectura general del ecosistema Praeco.

Como se muestra en la Figura 3.1, el ecosistema se divide en tres aplicaciones con responsabilidades diferenciadas. Esta separación resulta importante metodológicamente: Praeco Desarrollo permite desarrollar, aprender y evaluar el sistema; Praeco Usuario representa la aplicación destinada al uso normal; y Praeco Análisis estudia experimentalmente los resul-

tados obtenidos.

3.1.1. Flujo general de procesamiento

El funcionamiento principal de Praeco sigue una cadena secuencial: adquisición de la imagen, selección de la región de interés, evaluación de calidad, reconocimiento OCR, filtrado, reconstrucción y protección de tokens, corrección léxica, detección de alérgenos y síntesis de voz.

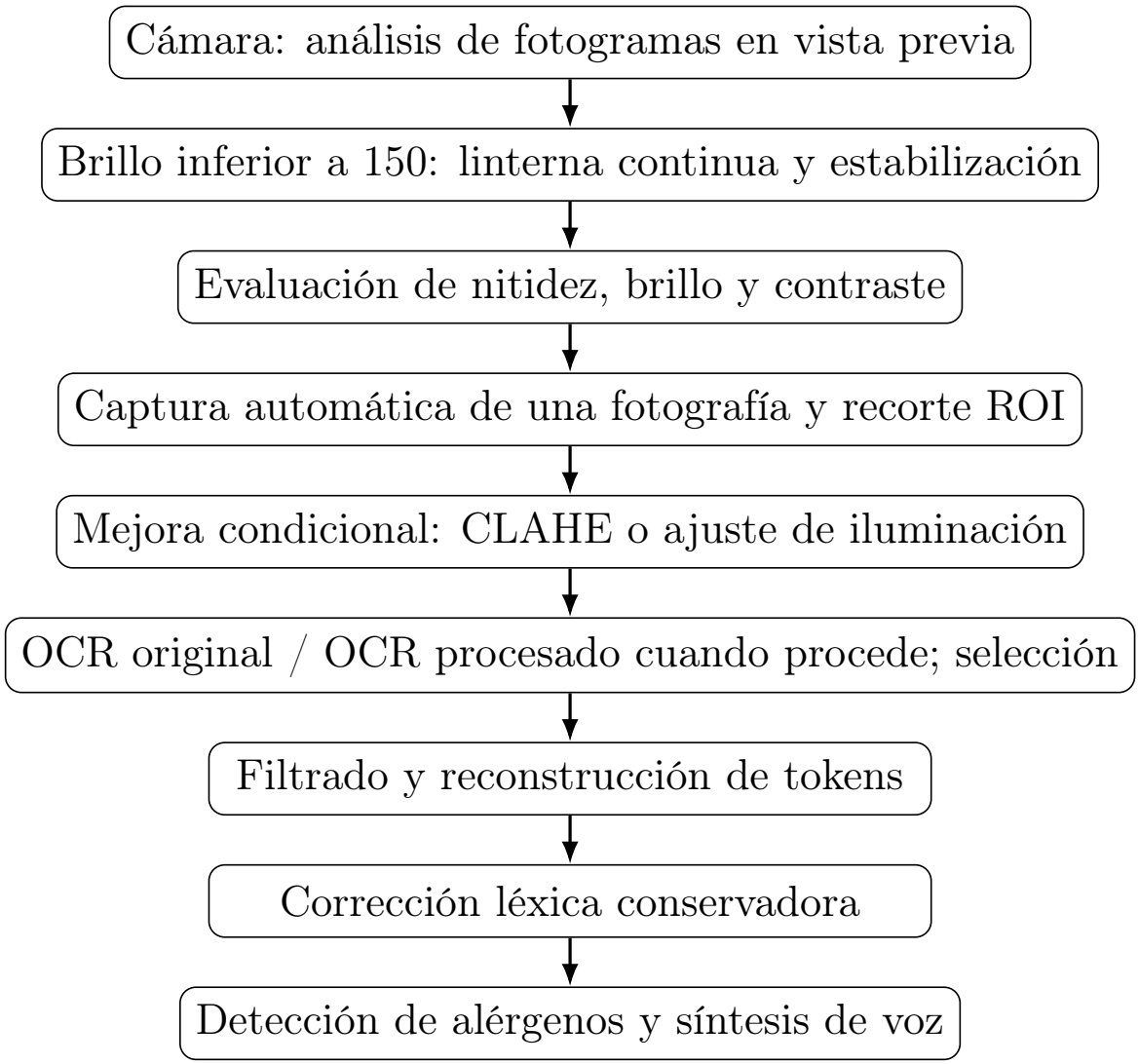


Figure 3.2: Flujo de captura rápida y preprocesamiento condicional de Praeco. La captura se realiza al cumplir los criterios de calidad; la linterna permanece encendida hasta finalizarla cuando es necesaria.

La Figura 3.2 resume el flujo de procesamiento desde la captura hasta la obtención y comunicación del resultado.

3.1.2. Adquisición de la imagen y región de interés

El procesamiento comienza con la cámara del teléfono móvil. La cámara proporciona imágenes de la etiqueta situada delante del dispositivo. El objetivo no es únicamente tomar una fotografía, sino obtener una captura que presente condiciones suficientemente buenas para realizar posteriormente el reconocimiento OCR.

En la versión optimizada, Praeco analiza fotogramas de la vista previa antes de efectuar una única fotografía. La decisión automática se basa en la estabilidad de la imagen y en estimaciones de nitidez, brillo y contraste. Se evita así adquirir y reconocer mediante OCR tres fotografías sucesivas para escoger una de ellas, operación que aumentaba la latencia. Cuando la imagen no reúne los criterios establecidos, la aplicación continúa evaluando la vista previa hasta obtener condiciones apropiadas.

Una vez realizada la captura, Praeco recorta una *Region of Interest* (ROI), que delimita la zona de la imagen que interesa procesar. En la implementación se conserva todo el ancho y aproximadamente los dos tercios superiores de la imagen. Esta reducción permite limitar la presencia de fondo y de elementos que no pertenecen al etiquetado, al mismo tiempo que disminuye el volumen de información que debe procesarse.

La imagen correspondiente a la ROI constituye la entrada del motor OCR. Se conserva una copia original y, cuando las condiciones lo requieren, se genera otra con mejora de imagen. El reconocimiento se aplica inicialmente a la imagen original y puede compararse con el de la procesada, seleccionando la alternativa mediante los criterios de puntuación programados.

3.1.3. Evaluación de la calidad de imagen

Antes de aceptar una captura, Praeco analiza tres propiedades principales: **nitidez**, **brillo** y **contraste**. La nitidez permite estimar si la fotografía está correctamente enfocada y se calcula mediante la varianza de la respuesta de un operador Laplaciano. El brillo representa el nivel medio de luminancia, mientras que el contraste representa la dispersión de los niveles de luminancia.

La luminancia se aproxima mediante

$$L = 0{,}299R + 0{,}587G + 0{,}114B. \tag{3.1}$$

La Figura 3.3 representa las variables utilizadas para decidir si la imagen presenta unas condiciones mínimas de calidad. Los umbrales básicos de aceptación son nitidez mínima 100, contraste mínimo 20, brillo mínimo 35 y brillo máximo 225. Estos umbrales de aceptación se distinguen del umbral de activación de la linterna (brillo inferior a 150) y del criterio de CLAHE (contraste inferior a 25). Estos valores son parámetros iniciales del prototipo y no deben interpretarse como límites universalmente validados. Una imagen puede superar

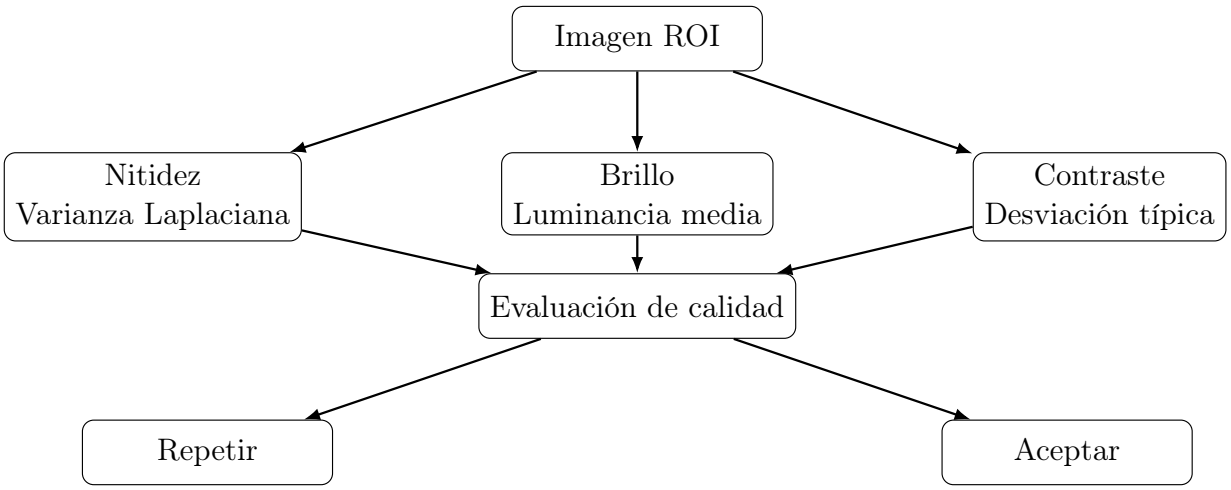


Figure 3.3: Evaluación de calidad de la imagen.

dichos umbrales y producir un OCR deficiente debido a la perspectiva, curvatura del envase, reflejos, tipografía, tamaño de letra o disposición del texto.

3.1.4. Control automático de iluminación y estabilización

La versión optimizada incorpora control de la iluminación auxiliar a partir de la luminancia estimada en la vista previa. Se ha fijado experimentalmente un umbral de activación de **150** en la escala de brillo de 0 a 255. Cuando la iluminación es inferior, se solicita al controlador de cámara el modo `FlashMode.torch`, que proporciona iluminación continua en lugar de un destello sincronizado con la fotografía. Una vez encendida, la linterna permanece activa durante la evaluación y la adquisición, incluso si el brillo aumenta por efecto de la propia luz. Esta retención evita la oscilación encendido–apagado observada durante el desarrollo.

Tras la activación se introduce una espera aproximada de **900 ms** para permitir la estabilización de la exposición y el enfoque. Finalizada la captura, se apaga la linterna. El umbral de 150 y el tiempo de espera son parámetros de ingeniería ajustados durante las pruebas del prototipo, no constantes óptimas universales: pueden depender de la cámara, la distancia y la reflectancia del envase.

3.1.5. Preprocesamiento condicional mediante CLAHE

Se ha añadido un módulo opcional de mejora de imagen previo al OCR. Su objetivo es actuar únicamente cuando la calidad de la captura lo justifique, conservando la imagen original como referencia. En particular, si el contraste medido es inferior a un umbral operativo de **25**, se puede aplicar *Contrast Limited Adaptive Histogram Equalization* (CLAHE), una ecualización adaptativa del histograma limitada para reducir la amplificación excesiva del ruido.

El procedimiento divide la imagen en regiones locales, calcula sus histogramas de luminancia, limita las frecuencias excesivas y redistribuye los valores de intensidad; la interpolación entre regiones reduce las discontinuidades de contraste. Si el problema dominante es la iluminación, se contempla un ajuste de brillo. Cuando la imagen ya presenta condiciones suficientes, no se aplica mejora adicional. CLAHE no corrige por sí mismo el desenfoque ni garantiza una disminución del error OCR.

La interfaz de resultados presenta la fotografía original y, cuando se ha generado, la procesada, junto con la indicación del tratamiento utilizado (CLAHE, ajuste de iluminación o ninguno) y la procedencia de la imagen elegida para el reconocimiento. Esta comparación visual permite comprobar el efecto de la mejora sin confundirlo con una ganancia de precisión demostrada.

La selección entre resultados OCR se efectúa mediante la puntuación prevista en la implementación y no mediante el *ground truth*, que se reserva para la evaluación experimental. La incorporación de CLAHE debe evaluarse mediante un ensayo de ablación con imágenes idénticas y los mismos recursos lingüísticos congelados.

3.1.6. Reconocimiento OCR mediante Google ML Kit

Una vez aceptada la imagen, se procesa mediante `TextRecognizer` de Google ML Kit. La salida inicial constituye el **OCR bruto**. Esta salida puede contener sustituciones de caracteres visualmente parecidos, errores de separación, puntuación o reconocimiento de palabras. Por ello, ML Kit constituye la primera etapa del reconocimiento textual, pero no el resultado definitivo utilizado por Praeco.

3.1.7. Filtrado del OCR

El texto bruto pasa por una etapa de filtrado cuyo objetivo es eliminar información que no resulta útil para el análisis y reducir la interferencia de determinados elementos comerciales o auxiliares. El filtro trabaja por líneas y puede descartar URL, cadenas exclusivamente numéricas, determinados códigos largos, identificadores de lote, cantidades aisladas con unidades y ciertas expresiones de reciclaje. Las líneas que contienen información textual útil se conservan para las etapas posteriores.

Debe distinguirse entre **OCR bruto**, **OCR filtrado** y **texto corregido**. El primero es la salida directa del motor OCR; el segundo es el resultado de aplicar las reglas de limpieza; y el tercero incorpora el postprocesamiento léxico de Praeco.

3.1.8. Reconstrucción y protección de tokens

Tras el filtrado, el texto se representa como una secuencia de palabras o *tokens*. Esta representación permite analizar cada palabra antes de decidir si debe mantenerse o corregirse.

Praeco incorpora además mecanismos para reconstruir determinadas separaciones o uniones anómalas producidas por el OCR.

Uno de los principios del corrector es que una palabra desconocida no debe corregirse automáticamente. Antes de aplicar una corrección se comprueba si el token pertenece a alguna categoría protegida: números y cantidades, palabras muy cortas, *stopwords*, vocabulario presente en el diccionario base o adaptativo, determinados términos ingleses y nombres propios explícitamente protegidos.

Esta protección es especialmente relevante en etiquetas multilingües. Por ejemplo, la palabra italiana *bianco* puede aparecer realmente en una etiqueta y, si forma parte del vocabulario válido aprendido, debe conservarse. El objetivo del corrector no es traducir ni castellanizar el etiquetado, sino corregir errores atribuibles al OCR.

3.1.9. Corrector léxico híbrido

Cuando un token no está protegido, Praeco genera candidatos a partir del diccionario base, del diccionario adaptativo y de los recursos léxicos disponibles. Cada candidato se evalúa combinando distancia de edición, semejanza fonética y diferencia de longitud. Si el mejor candidato no alcanza una confianza suficiente o existe ambigüedad entre candidatos, se conserva la palabra original.

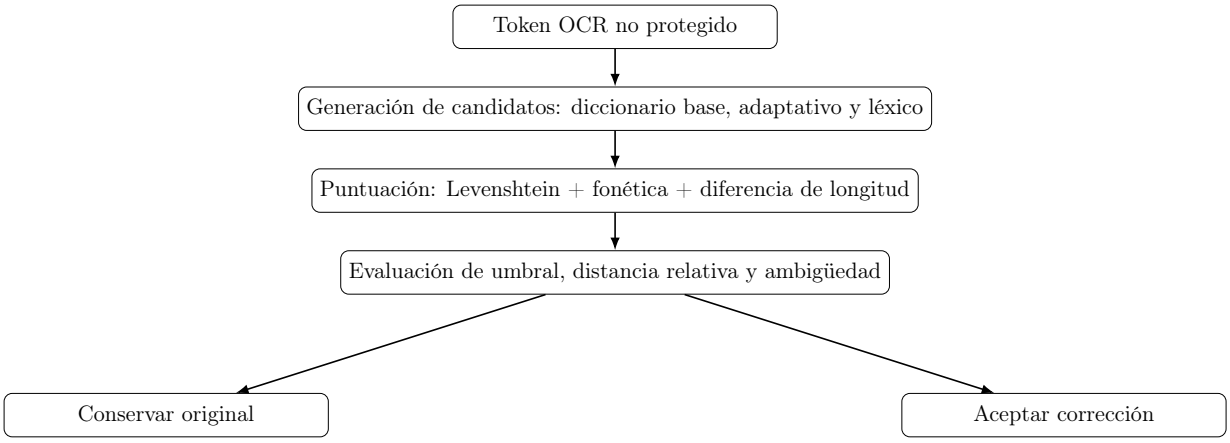


Figure 3.4: Proceso de decisión del corrector léxico de Praeco.

La lógica general de decisión del corrector se sintetiza en la Figura 3.4.

3.1.10. Distancia de Levenshtein

La Distancia de Levenshtein es uno de los algoritmos más utilizados en procesamiento del lenguaje natural (NLP), corrección ortográfica y sistemas OCR para medir la similitud entre dos palabras.

La distancia de Levenshtein mide el número mínimo de operaciones necesarias para transformar una cadena en otra mediante inserciones, eliminaciones y sustituciones. Por ejemplo,

transformar `lechf` en `leche` requiere una sustitución, por lo que su distancia es uno. Una distancia pequeña indica elevada similitud ortográfica, pero Praeco no utiliza este valor de forma aislada: se combina con otros criterios para evitar correcciones agresivas.

Los límites de distancia dependen de la longitud de la palabra: para tokens de longitud menor o igual que cinco se admite como máximo una distancia ponderada de 1; hasta ocho caracteres, 2; y para longitudes superiores, 3. Además se aplica un límite relativo y criterios de confianza y separación entre candidatos.

3.1.11. Información fonética

La semejanza fonética complementa la semejanza ortográfica. La función de fonetización agrupa algunas grafías potencialmente confundibles y permite penalizar candidatos que, aunque sean próximos gráficamente, resulten menos plausibles desde el punto de vista fonético. La decisión final combina ambas fuentes de información con la diferencia de longitud.

3.1.12. Diccionario adaptativo y aprendizaje incremental

Praeco Desarrollo incorpora un mecanismo de aprendizaje basado en dos recursos diferentes. El fichero `errores_ocr.json` registra asociaciones observadas entre formas reconocidas erróneamente y palabras correctas, junto con su frecuencia y estado. El fichero `diccionario_adaptativo.json`, en cambio, contiene vocabulario válido aprendido y no una tabla de errores.

Cuando el modo de aprendizaje está activo, el texto OCR se alinea con el texto real o *ground truth*. Las asociaciones próximas se registran y aumentan su frecuencia cuando vuelven a observarse. Las asociaciones suficientemente repetidas, válidas y no conflictivas pueden integrarse en el diccionario adaptativo. En la implementación actual, la integración exige una frecuencia mínima de tres observaciones.

La Figura 3.5 muestra la secuencia seguida durante el aprendizaje incremental, desde la comparación con el texto de referencia hasta la posible incorporación de vocabulario validado.

Por ejemplo, si el OCR produce repetidamente *biarico* mientras el texto real contiene *bianco*, la asociación puede registrarse como error. En cambio, *bianco* puede incorporarse al diccionario adaptativo como palabra válida. De esta forma, el sistema aprende vocabulario del dominio sin considerar erróneas palabras extranjeras que realmente aparecen en el etiquetado.

3.1.13. Corrección contextual mediante n-gramas

El proyecto dispone de recursos de unigramas, bigramas, trigramas y cuatrigramas relacionados con el dominio alimentario. Estos recursos permiten representar secuencias fre-

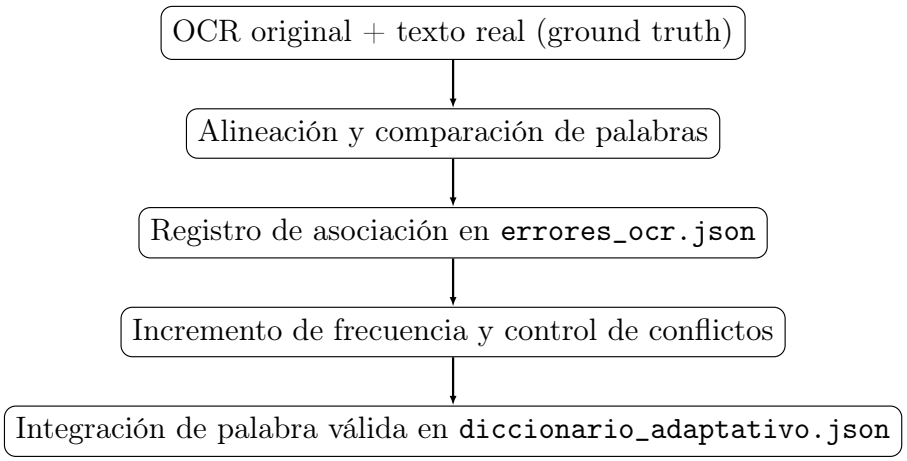


Figure 3.5: Esquema del aprendizaje incremental de Praeco Desarrollo.

cuentes como *aceite de*, *harina de*, *puede contener* o *trazas de*. El contexto puede reforzar una corrección que resulte coherente con las palabras vecinas.

No obstante, en la validación congelada utilizada para los resultados finales debe indicarse expresamente si la llamada a la corrección contextual está activa o desactivada. Cuando se encuentra desactivada, los resultados corresponden al OCR y al corrector léxico activo y no constituyen una evaluación de la corrección contextual por n-gramas.

3.1.14. Detección de alérgenos

El texto corregido se utiliza como entrada del detector de alérgenos. La implementación contempla las categorías gluten, lácteos, huevo, frutos secos, cacahuete, soja, pescado y marisco mediante un conjunto de términos configurados para cada una de ellas. El sistema analiza la presencia de estos términos en el texto reconocido y, cuando identifica una coincidencia positiva, genera un aviso del tipo *«Atención: contiene ...»*.

Para reducir la aparición de falsos positivos, Praeco no considera únicamente la presencia aislada del nombre de un alérgeno, sino que tiene en cuenta determinadas expresiones que indican explícitamente su ausencia. De este modo, una aparición de la palabra *gluten* no implica necesariamente que el producto contenga gluten. Si el texto reconocido contiene expresiones como *«sin gluten»*, *«libre de gluten»* o construcciones equivalentes contempladas por las reglas implementadas, el sistema interpreta que dicha mención corresponde a una declaración de ausencia y evita generar una alerta positiva para ese alérgeno. El mismo criterio puede aplicarse a otras categorías cuando aparecen asociadas a expresiones de exclusión reconocidas por el sistema.

Por tanto, el proceso de detección puede representarse mediante dos niveles de comprobación. En primer lugar, se localizan los términos asociados a las distintas categorías de alérgenos y, posteriormente, se comprueba si la aparición se encuentra dentro de una expresión de exclusión. Si existe una coincidencia positiva y no está afectada por una regla de

ausencia, Praeco genera el mensaje *«Atención: contiene ...»*. Si no se detectan términos configurados como alérgenos, o las apariciones encontradas corresponden únicamente a expresiones explícitas de ausencia reconocidas por el sistema, se informa mediante el mensaje *«No se detectan alérgenos»*.

Este tratamiento resulta especialmente importante en el etiquetado alimentario, ya que expresiones como *«sin gluten»* contienen literalmente el término que identifica al alérgeno. Una búsqueda basada exclusivamente en palabras clave produciría en este caso un falso positivo. La comprobación de expresiones de ausencia permite a Praeco distinguir, dentro de las reglas implementadas, entre una mención positiva y una declaración explícita de exclusión.

No obstante, este mecanismo constituye una ayuda experimental basada en coincidencias léxicas y reglas de contexto local, y no un análisis semántico, nutricional o clínico completo. La expresión *«No se detectan alérgenos»* significa únicamente que, a partir del texto reconocido por el OCR y de las reglas y categorías configuradas en Praeco, no se ha identificado una presencia positiva de los alérgenos considerados. Por ello, este resultado no certifica que el producto esté libre de alérgenos y no debe utilizarse como única fuente para decidir su consumo.

3.1.15. Síntesis de voz

La accesibilidad se completa mediante *Text-to-Speech* (TTS). El texto corregido y los avisos generados se transforman en voz, permitiendo utilizar la aplicación sin depender exclusivamente de la lectura visual de la pantalla. La configuración principal utiliza el idioma `es-ES`.

La velocidad de reproducción de la síntesis de voz es un parámetro configurable mediante software, lo que permite adaptar el ritmo de lectura a las necesidades y preferencias del usuario. De esta forma, la aplicación puede proporcionar una locución más lenta para facilitar la comprensión o, alternativamente, aumentar la velocidad para usuarios habituados al uso de lectores de pantalla y sistemas de asistencia por voz. Esta posibilidad de configuración contribuye a mejorar la accesibilidad y flexibilidad del sistema, al permitir personalizar la presentación auditiva de la información. La velocidad programada por defecto es 0,5.

3.1.16. Evaluación mediante ground truth, CER y WER

Esta funcionalidad está presente solo en Praeco Desarrollo.

Durante los experimentos, el usuario introduce el texto real de la etiqueta, que actúa como *ground truth*. La referencia se compara tanto con el OCR original como con el texto corregido.

El *Character Error Rate* (CER) cuantifica el error a nivel de caracteres:

$$CER = \frac{S + D + I}{N}, \tag{3.2}$$

donde S representa sustituciones, D eliminaciones, I inserciones y N el número de caracteres de la referencia.

El *Word Error Rate* (WER) utiliza la misma expresión a nivel de palabras:

$$WER = \frac{S + D + I}{N}, \tag{3.3}$$

donde N representa ahora el número de palabras de la referencia. La comparación de `cer_ocr` frente a `cer_corregido` y de `wer_ocr` frente a `wer_corregido` permite determinar objetivamente si el corrector mejora, mantiene o empeora el reconocimiento.

3.1.17. Medición temporal

Praeco Desarrollo instrumenta las principales fases del procesamiento mediante temporizadores. Se registran el tiempo de captura $t_{captura}$, el tiempo OCR t_{OCR}, el tiempo de corrección $t_{correccion}$ y el tiempo de detección de alérgenos $t_{alergenos}$. El tiempo total analizado puede obtenerse como

$$t_{total} = t_{captura} + t_{OCR} + t_{correccion} + t_{alergenos}. \tag{3.4}$$

En la versión de captura rápida, la medida de captura debe interpretarse de acuerdo con los puntos de inicio y fin del temporizador: no necesariamente incluye todas las operaciones posteriores de decodificación, recorte y mejora de imagen. Para comparar versiones se recomienda registrar además la latencia completa desde la activación hasta el resultado. Estas medidas permiten evaluar no solo la precisión, sino también el coste computacional y la viabilidad del procesamiento en tiempo real.

3.1.18. Almacenamiento experimental

Praeco Desarrollo crea y en caso de existir actualiza un fichero canónico denominado `experimentos.csv`. El registro de referencia utilizado para la validación histórica contiene 21 campos:

1. `fecha`	2. `imagen`	3. `texto_real`
4. `ocr`	5. `ocr_corregido`	6. `cer_ocr`
7. `cer_corregido`	8. `wer_ocr`	9. `wer_corregido`
10. `ancho`	11. `alto`	12. `nitidez`
13. `brillo`	14. `contraste`	15. `calidad_aceptable`
16. `decision_calidad`	17. `t_captura`	18. `t_ocr`
19. `t_correccion`	20. `t_alergenos`	21. `modo_aprendizaje`

La instrumentación futura del control de iluminación y del preprocesamiento puede ampliar este esquema con los campos `brillo_previo`, `linterna_activada`, `tiempo_estabilizacion_ms` y `mejora_imagen`. No se presupone que estén presentes en el CSV histórico. Además de este fichero CSV, la versión de Desarrollo conserva información de trazabilidad y recursos de aprendizaje. El campo `modo_aprendizaje` permite distinguir las ejecuciones realizadas durante el aprendizaje de aquellas efectuadas con los recursos congelados para validación.

3.1.19. Praeco Análisis

Praeco Análisis constituye una aplicación independiente destinada a procesar el fichero `experimentos.csv`. El usuario selecciona la ubicación del fichero de experimentos y la aplicación calcula estadísticas globales, compara CER y WER antes y después de la corrección, analiza tiempos de ejecución y permite estudiar las variables de calidad. Esta separación evita incorporar herramientas experimentales innecesarias en la aplicación destinada al usuario final.

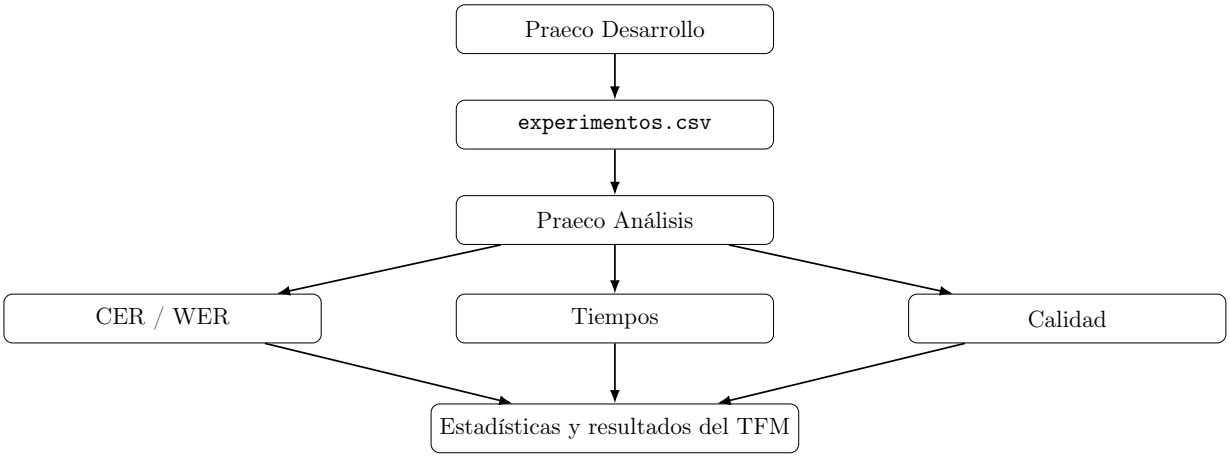


Figure 3.6: Relación entre Praeco Desarrollo y Praeco Análisis.

La relación entre la aplicación que genera los datos experimentales y la herramienta que los analiza se resume en la Figura 3.6.

3.1.20. Separación entre Praeco Desarrollo y Praeco Usuario

La separación entre ambas versiones es fundamental para la metodología experimental. Praeco Desarrollo incorpora *ground truth*, cálculo de CER/WER, aprendizaje, almacenamiento experimental y trazabilidad. Praeco Usuario conserva únicamente las funciones necesarias para el uso normal: cámara, control de calidad, OCR, filtrado, corrección con recursos congelados, detección de alérgenos y TTS.

El procedimiento metodológico consiste en aprender y depurar los recursos en Desarrollo, auditar el *ground truth*, congelar el código y los recursos de aprendizaje, realizar la validación independiente y transferir el diccionario adaptativo validado a la aplicación de Usuario. De este modo, los resultados de aprendizaje no se mezclan con los resultados de validación final.

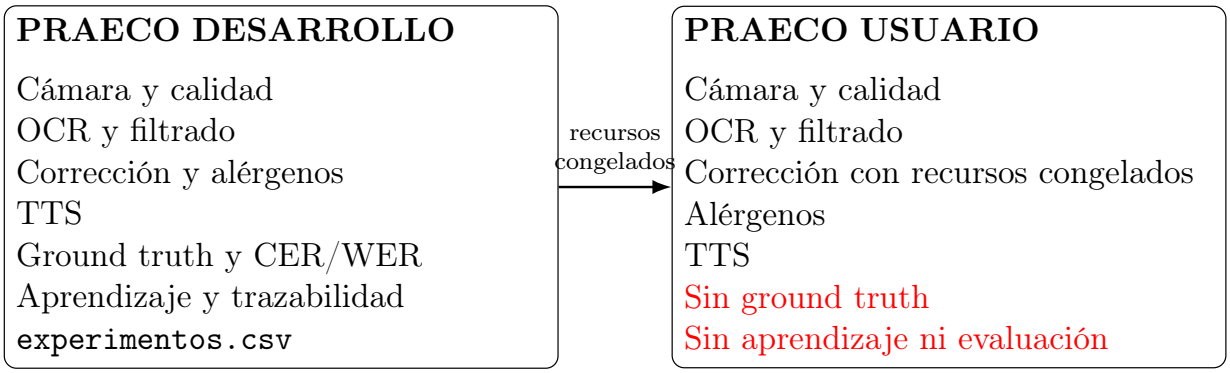


Figure 3.7: Separación funcional entre Praeco Desarrollo y Praeco Usuario.

La Figura 3.7 destaca la separación entre las funciones experimentales de Praeco Desarrollo y la versión destinada al uso normal. En conjunto, Praeco implementa la secuencia **adquisición de una imagen de calidad → reconocimiento OCR → filtrado y reconstrucción → corrección híbrida → detección de alérgenos → comunicación accesible mediante voz**. La versión de Desarrollo añade paralelamente aprendizaje, trazabilidad y evaluación cuantitativa mediante CER, WER y tiempos de procesamiento.

3.2. Tecnologías y Dependencias Utilizadas

Las principales bibliotecas y herramientas empleadas son:

- **Flutter**: Framework de UI para el desarrollo de aplicaciones móviles multiplataforma.
- **camera**: Plugin de Flutter para acceder y controlar la cámara del dispositivo.
- **google_mlkit_text_recognition**: Parte de Google ML Kit, utilizada para realizar el reconocimiento óptico de caracteres en las imágenes capturadas.
- **flutter_tts**: Plugin de Flutter para la funcionalidad de texto a voz (*Text-to-Speech*), permitiendo a la aplicación vocalizar el texto reconocido y corregido.

- **image (package image)**: Biblioteca para el procesamiento de imágenes en Dart, utilizada para recortar las imágenes antes de pasarlas al motor OCR.
- **path_provider**: Plugin para obtener rutas a directorios comúnmente utilizados en el sistema de archivos.
- **permission_handler**: Plugin para gestionar los permisos de la aplicación, como el acceso al almacenamiento.
- **google_mlkit_language_id**: Parte de Google ML Kit, utilizada para identificar el idioma del texto detectado y filtrar contenido no relevante.

3.3. Justificación de la elección de Flutter y Dart

Uno de los aspectos fundamentales durante el desarrollo del sistema fue la selección de las tecnologías software más adecuadas para implementar un asistente inteligente accesible y multiplataforma.

Tras analizar diversas alternativas, se decidió utilizar Flutter junto al lenguaje Dart como base principal del desarrollo.

- **Motivo de la elección.** La elección de Flutter se fundamenta principalmente en los siguientes aspectos:
  - Desarrollo multiplataforma mediante un único código fuente.
  - Alto rendimiento en dispositivos móviles.
  - Integración sencilla con librerías de visión artificial y OCR.
  - Excelente compatibilidad con Android.
  - Interfaz gráfica moderna y accesible.
  - Gran ecosistema de paquetes y librerías.
  - Rapidez de desarrollo y mantenimiento.

Flutter permite generar aplicaciones nativas con un rendimiento cercano al desarrollo específico para Android o iOS, reduciendo considerablemente el tiempo de implementación y facilitando futuras ampliaciones del sistema.

- **Comparación con otras tecnologías.** Durante la fase de diseño se analizaron otras alternativas tecnológicas ampliamente utilizadas en el desarrollo móvil:
  - Android Studio (Java/Kotlin).
  - React Native.
  - Xamarin.
  - Desarrollo web híbrido.

- **Android Studio con Java o Kotlin.** El desarrollo nativo en Android Studio proporciona un excelente rendimiento y acceso directo al hardware del dispositivo. Sin embargo, presenta diversos inconvenientes para este proyecto:

  - Mayor complejidad de desarrollo.
  - Código más extenso.
  - Menor velocidad de prototipado.
  - Dificultad para mantener versiones multiplataforma.

Flutter permitió implementar funcionalidades complejas de visión artificial y accesibilidad utilizando una arquitectura considerablemente más compacta y mantenible.

- **React Native** constituye otra alternativa multiplataforma ampliamente utilizada. No obstante, se descartó debido a:

  - Menor rendimiento gráfico en tiempo real.
  - Dependencia elevada de puentes nativos.
  - Integración más compleja con procesamiento de cámara y OCR.
  - Mayor latencia en aplicaciones intensivas de visión artificial.

Dado que el proyecto requiere captura continua de imagen, procesamiento OCR y síntesis de voz en tiempo real, Flutter ofrecía un comportamiento más estable y eficiente.

- **Xamarin** proporciona integración con tecnologías Microsoft y desarrollo multiplataforma basado en C#. Sin embargo, se descartó debido a:

  - Menor comunidad activa.
  - Menor cantidad de librerías especializadas en IA móvil.
  - Mayor complejidad de despliegue.
  - Menor documentación orientada a visión artificial móvil.

3.4. Ventajas específicas de Flutter en este proyecto

Flutter aporta ventajas especialmente relevantes para el desarrollo del asistente inteligente:

- Integración directa con Google ML Kit.
- Compatibilidad sencilla con Flutter TTS.
- Alto rendimiento en captura de cámara.
- Arquitectura reactiva adecuada para procesamiento asíncrono.
- Interfaz accesible y adaptable.
- Desarrollo rápido de prototipos.

Además, Dart ofrece un sistema moderno de programación asíncrona basado en `Future` y `async/await`, especialmente útil para coordinar tareas concurrentes como:

- Captura de imagen.
- Procesamiento OCR.
- Análisis contextual.
- Síntesis de voz.

3.5. Conclusión tecnológica

La combinación Flutter + Dart resultó especialmente adecuada para el desarrollo del sistema propuesto, proporcionando un equilibrio óptimo entre:

- Rendimiento.
- Accesibilidad.
- Escalabilidad.
- Facilidad de mantenimiento.
- Integración con inteligencia artificial.

La elección tecnológica permitió implementar un prototipo funcional avanzado con características próximas a asistentes visuales comerciales modernos.

Chapter 4

Resultados

4.1. Introducción

La evaluación experimental de Praeco se ha realizado de forma progresiva durante las distintas etapas de desarrollo. Los primeros ensayos permitieron caracterizar el comportamiento del OCR y detectar limitaciones del corrector inicial. Posteriormente se incorporaron reglas de protección, vocabulario adaptativo y un mecanismo de aprendizaje incremental. Finalmente, se estableció una metodología que separa de forma explícita los experimentos realizados con aprendizaje activo de aquellos ejecutados con los recursos lingüísticos congelados.

Esta separación es necesaria para interpretar correctamente los resultados. La variable `modo_aprendizaje`, almacenada en cada experimento, identifica las dos condiciones de funcionamiento:

- `modo_aprendizaje = true`: Praeco se encuentra en fase de desarrollo y aprendizaje. El *ground truth* puede utilizarse para registrar errores recurrentes del OCR y actualizar los recursos adaptativos cuando se cumplen los criterios establecidos.
- `modo_aprendizaje = false`: Praeco utiliza los recursos lingüísticos disponibles, pero no los modifica. Esta configuración se emplea para la validación con el conocimiento congelado, evitando que las muestras evaluadas alteren simultáneamente el comportamiento del corrector.

Por tanto, los resultados obtenidos en ambas modalidades no deben mezclarse como si pertenecieran a una única condición experimental. Los ensayos con aprendizaje activo permiten estudiar la construcción del conocimiento adaptativo, mientras que los realizados con aprendizaje desactivado permiten evaluar el comportamiento del sistema con una configuración fija.

La versión final del trabajo utiliza además Praeco Análisis como aplicación independiente para procesar los experimentos generados por Praeco Desarrollo y estudiar CER, WER, tiempos, variables de calidad y diferencias entre las dos modalidades de aprendizaje.

4.2. Métricas de evaluación

4.2.1. Character Error Rate (CER)

La métrica CER (*Character Error Rate*) cuantifica el error a nivel de caracteres respecto al texto real o *ground truth*. Se calcula mediante:

$$\mathrm{CER} = \frac{S + D + I}{N} \tag{4.1}$$

donde S es el número de sustituciones, D el número de eliminaciones, I el número de inserciones y N el número de caracteres del texto de referencia. El cálculo se basa en la distancia de Levenshtein. Un valor de CER igual a cero representa una coincidencia perfecta.

4.2.2. Word Error Rate (WER)

La métrica WER (*Word Error Rate*) aplica el mismo principio a nivel de palabras:

$$\mathrm{WER} = \frac{S + D + I}{N} \tag{4.2}$$

donde, en este caso, S, D e I representan sustituciones, eliminaciones e inserciones de palabras y N es el número de palabras del texto de referencia. CER y WER se calculan tanto para la salida OCR de referencia como para el texto obtenido después de aplicar el corrector de Praeco.

4.2.3. Criterio de comparación

Para cuantificar el efecto del corrector se utilizan las diferencias:

$$\Delta\mathrm{CER} = \mathrm{CER}_{\mathrm{CORR}} - \mathrm{CER}_{\mathrm{OCR}} \tag{4.3}$$

$$\Delta\mathrm{WER} = \mathrm{WER}_{\mathrm{CORR}} - \mathrm{WER}_{\mathrm{OCR}} \tag{4.4}$$

Un valor negativo indica una reducción del error, un valor igual a cero indica que la corrección no ha modificado la métrica y un valor positivo indica un empeoramiento. Este criterio permite incluir también los casos en los que el OCR original presenta error cero; estos experimentos son especialmente relevantes para detectar si el corrector modifica de forma incorrecta un texto que ya era correcto.

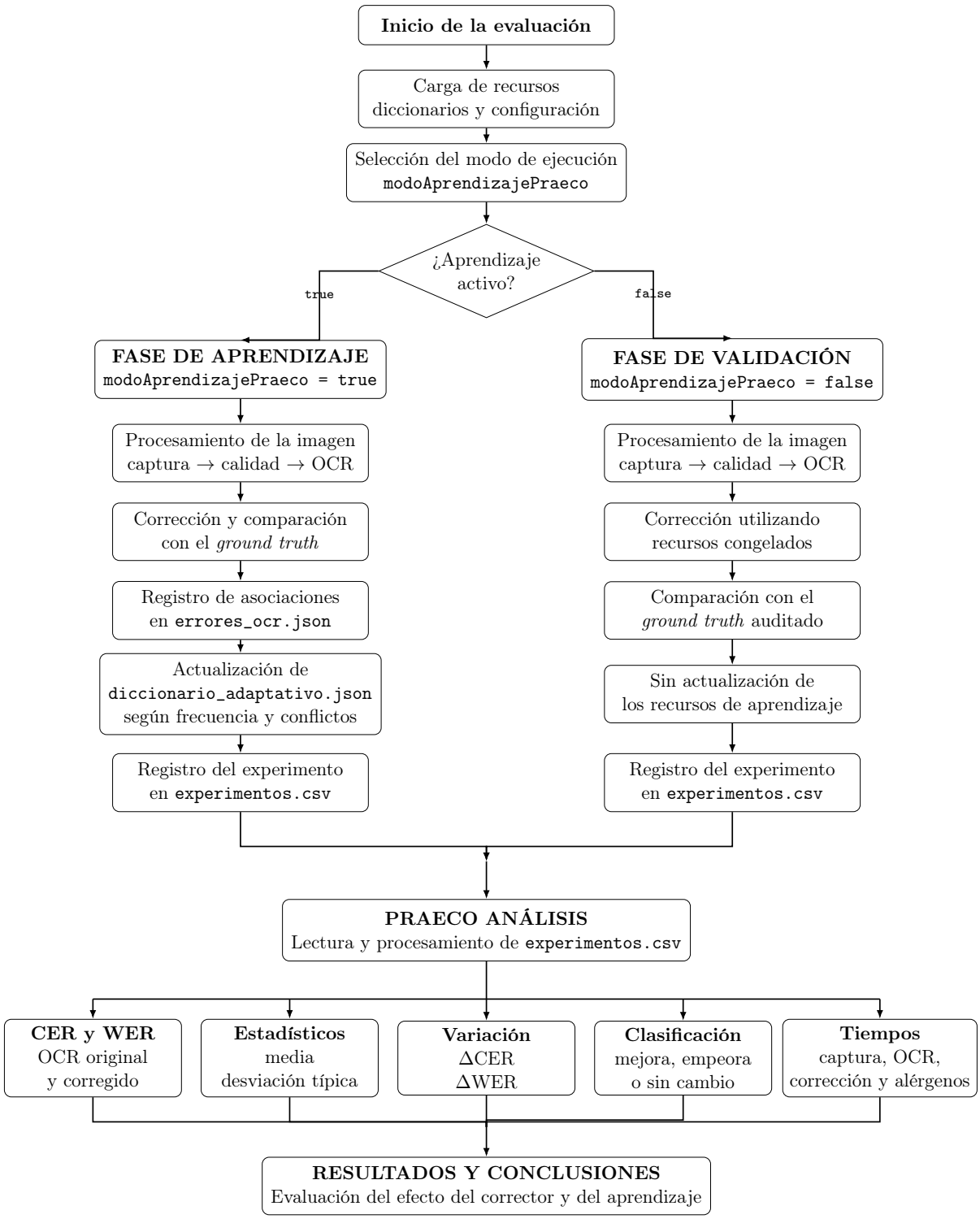


Figure 4.1: Flujo de evaluación experimental de Praeco.

Los experimentos con aprendizaje activo permiten actualizar los recursos adaptativos, mientras que la validación se realiza con dichos recursos congelados. Los resultados registrados son posteriormente procesados mediante Praeco Análisis.

4.3. Resultados preliminares

En una primera etapa se realizaron cinco experimentos con etiquetas reales de productos alimentarios. Estos resultados corresponden a una versión anterior del corrector y se mantienen en la memoria porque fueron determinantes para identificar los problemas que posteriormente motivaron la incorporación de reglas de protección y aprendizaje adaptativo.

Table 4.1: Resultados de los cinco experimentos preliminares.

Exp.	CER OCR	CER Corr.	WER OCR	WER Corr.
1	0.1969	0.2073	0.2778	0.3333
2	0.1265	0.1344	0.2162	0.2432
3	0.0941	0.0941	0.2895	0.2632
4	0.0407	0.0407	0.2174	0.2174
5	0.1165	0.1225	0.1829	0.2195

Los valores medios fueron:

$$\overline{\mathrm{CER}}_{\mathrm{OCR}} = 0.1149, \qquad \overline{\mathrm{CER}}_{\mathrm{CORR}} = 0.1198 \tag{4.5}$$

$$\overline{\mathrm{WER}}_{\mathrm{OCR}} = 0.2368, \qquad \overline{\mathrm{WER}}_{\mathrm{CORR}} = 0.2553 \tag{4.6}$$

Por tanto, en esta primera versión el corrector no produjo una mejora global. Sin embargo, el análisis individual mostró comportamientos diferentes. En el experimento 3, por ejemplo, el WER disminuyó de 0.2895 a 0.2632, mientras que en otros experimentos aparecieron sustituciones incorrectas que incrementaron el error.

Estos resultados fueron útiles para orientar el desarrollo hacia un corrector más conservador: si una sustitución no presenta suficiente confianza, es preferible conservar el término reconocido por el OCR antes que introducir una modificación potencialmente errónea.

4.4. Evolución del corrector

A partir de los resultados preliminares se incorporaron mecanismos destinados a reducir las correcciones espurias. Entre ellos se encuentran la protección de números y unidades, palabras cortas, *stopwords*, términos presentes en los diccionarios, determinados nombres propios y vocabulario específico del dominio alimentario.

También se implementó un sistema de aprendizaje incremental. Durante la fase de aprendizaje, Praeco compara el OCR con el *ground truth*, registra asociaciones plausibles en `errores_ocr.json`, incrementa su frecuencia cuando vuelven a aparecer y controla posibles conflictos. Cuando se cumplen los criterios definidos, el término correcto puede incorporarse como vocabulario válido a `diccionario_adaptativo.json`.

El diccionario adaptativo no debe interpretarse como una tabla destinada a convertir cualquier término desconocido en la palabra española más próxima. Su función es almacenar vocabulario válido aprendido durante el desarrollo. Este comportamiento es especialmente importante en etiquetas multilingües. Por ejemplo, *bianco* puede aparecer legítimamente en una etiqueta italiana; si el término forma parte del texto real, debe conservarse y no sustituirse por *blanco*.

4.5. Ensayos con aprendizaje activo

Los experimentos ejecutados con:

```
modoAprendizajePraeco = true
```

corresponden a la fase de desarrollo y aprendizaje. En esta modalidad, Praeco puede utilizar el *ground truth* para actualizar sus recursos adaptativos después de guardar la evaluación.

En uno de los conjuntos de pruebas realizados durante la evolución de Praeco se analizaron cuatro capturas y se obtuvo una reducción del CER medio desde aproximadamente 0.03086 hasta 0.02587. Para WER, el valor medio pasó aproximadamente de 0.07563 a 0.04404. Expresado como reducción relativa del error, estos cambios corresponden aproximadamente a un 16.19% para CER y un 41.77% para WER.

Table 4.2: Resumen de un ensayo con aprendizaje activo.

Métrica	**OCR**	**Corregido**	**Reducción relativa**
CER	0.03086	0.02587	16.19%
WER	0.07563	0.04404	41.77%

En ese conjunto se identificaron tres correcciones efectuadas por Praeco y las tres resultaron correctas. El resultado muestra que el conocimiento adaptativo puede ser útil cuando las asociaciones aprendidas representan adecuadamente los errores encontrados.

No obstante, estos valores no deben interpretarse como una validación independiente de la capacidad de generalización. El aprendizaje se encuentra habilitado y el objetivo de esta fase es construir y depurar el conocimiento que posteriormente será congelado para la evaluación.

4.6. Validación con aprendizaje desactivado

Para separar el aprendizaje de la evaluación se realizaron posteriormente ensayos con:

```
modoAprendizajePraeco = false
```

En esta configuración, Praeco carga los recursos lingüísticos disponibles, pero no modifica `errores_ocr.json` ni `diccionario_adaptativo.json` a partir de las muestras evaluadas. De esta forma, el corpus de validación no altera simultáneamente el conocimiento utilizado por el corrector.

Durante esta etapa se construyó un corpus de 28 capturas correspondientes a diferentes productos. El procedimiento experimental incluyó la revisión de las imágenes originales y la auditoría manual del *ground truth*. Esta revisión es necesaria porque una transcripción de referencia incorrecta puede atribuir al OCR o al corrector un error que realmente no existe.

En una evaluación intermedia, correspondiente a 21 muestras cuyo *ground truth* ya había sido auditado en ese momento, se obtuvieron los siguientes valores medios:

$$\overline{\mathrm{CER}}_{\mathrm{OCR}} = 0.05909, \qquad \overline{\mathrm{CER}}_{\mathrm{CORR}} = 0.05981 \tag{4.7}$$

$$\overline{\mathrm{WER}}_{\mathrm{OCR}} = 0.19732, \qquad \overline{\mathrm{WER}}_{\mathrm{CORR}} = 0.20131 \tag{4.8}$$

La variación relativa fue aproximadamente de +1.21% para CER y +2.02% para WER. En consecuencia, en este subconjunto de validación congelada el corrector no produjo una mejora global, aunque la diferencia respecto al OCR original fue reducida.

Table 4.3: Evaluación intermedia con aprendizaje desactivado y *ground truth* auditado.

Métrica	**OCR**	**Corregido**	**Variación relativa**
CER	0.05909	0.05981	+1.21%
WER	0.19732	0.20131	+2.02%

Es importante señalar que estos valores corresponden al análisis intermedio de 21 muestras auditadas y no deben confundirse con el corpus de 28 capturas utilizado en una etapa posterior de auditoría. Ambas evaluaciones forman parte de etapas intermedias del desarrollo. Tras completar la depuración del *ground truth* y ampliar el conjunto experimental, se realizó la validación final con los recursos de aprendizaje congelados. El conjunto utilizado para los resultados globales finales de este trabajo está formado por 63 registros, todos ellos obtenidos con `modo_aprendizaje = false`. Salvo que se indique expresamente lo contrario, las métricas globales presentadas en la parte final de este capítulo corresponden a esos 63 registros.

Los ensayos con aprendizaje desactivado permitieron además localizar falsos positivos concretos del corrector y comprobar la necesidad de proteger vocabulario legítimo que puede no pertenecer al castellano. En estos casos, el criterio de referencia es siempre el texto realmente impreso en la etiqueta y no la proximidad de la palabra a un término español.

4.7. Importancia de la auditoría del *ground truth*

La validación puso de manifiesto que la calidad del *ground truth* condiciona directamente CER y WER. Una referencia incorrecta puede hacer que una lectura correcta del OCR se contabilice como error o que una modificación incorrecta del corrector parezca una mejora.

Por este motivo, la metodología final incorpora una auditoría manual del texto real mediante comparación directa con las imágenes originales. Esta precaución es especialmente relevante en etiquetas multilingües, nombres propios, marcas, abreviaturas, cantidades y expresiones poco frecuentes. La presencia de una palabra italiana, inglesa o francesa no constituye por sí misma un error del OCR si esa palabra aparece realmente en la etiqueta.

4.8. Interpretación conjunta de los modos de aprendizaje

Los dos valores de `modo_aprendizaje` representan objetivos experimentales distintos:

- Con `modo_aprendizaje = true`, el objetivo es **aprender y depurar**. El sistema puede utilizar el *ground truth* para registrar errores y ampliar de forma controlada el vocabulario adaptativo.
- Con `modo_aprendizaje = false`, el objetivo es **validar**. Los recursos permanecen congelados y las muestras de validación no pueden modificar el conocimiento utilizado durante la propia prueba.

Por ello, una mejora observada con aprendizaje activo demuestra que el sistema puede aprovechar conocimiento adaptativo, pero no constituye por sí sola una medida independiente de generalización. La validación con aprendizaje desactivado proporciona una evaluación metodológicamente más adecuada del comportamiento del corrector una vez fijados sus recursos.

La secuencia metodológica adoptada para la evaluación final puede resumirse como:

aprendizaje → auditoría → congelación de recursos → validación

4.9. Análisis de las causas de error

El análisis individual de los experimentos permitió identificar diferentes fuentes de error:

- errores generados por el OCR debido a desenfoque, iluminación, contraste, tamaño de los caracteres o características tipográficas de la etiqueta;
- palabras válidas no presentes inicialmente en los recursos lingüísticos;
- términos pertenecientes a otros idiomas que forman parte legítima del etiquetado;
- palabras cortas, para las que una pequeña distancia de edición puede producir candidatos incorrectos;
- códigos, cantidades, unidades y expresiones alfanuméricas que no deben tratarse como palabras ordinarias;
- correcciones ambiguas en las que varios candidatos presentan puntuaciones similares;
- errores severos del OCR para los que no existe un candidato suficientemente próximo y fiable.

Estos resultados justifican la estrategia conservadora adoptada: ante una corrección insuficientemente fiable, resulta preferible conservar el término reconocido por el OCR antes que introducir una sustitución potencialmente incorrecta.

4.10. Análisis temporal

Además de CER y WER, Praeco registra el tiempo empleado en las principales etapas del procesamiento. Entre las variables almacenadas se encuentran `t_captura`, `t_ocr`, `t_correccion` y `t_alergenos`. Estas medidas permiten estudiar el coste computacional de cada etapa y comprobar si la incorporación del corrector y del detector de alérgenos mantiene un comportamiento compatible con el uso interactivo de la aplicación.

El análisis temporal debe interpretarse de forma independiente de CER y WER: los tiempos describen el coste de procesamiento, mientras que CER y WER cuantifican la precisión textual. Praeco Análisis permite estudiar conjuntamente ambas dimensiones sin introducir lógica estadística adicional en la aplicación destinada al usuario final.

La Tabla 4.4 muestra un análisis temporal por producto realizado sobre el subconjunto de capturas agrupadas por las nueve etiquetas indicadas, con `modoAprendizajePraeco = false`. Los valores correspondientes a cada producto se obtienen promediando las capturas disponibles para esa etiqueta. Por tanto, la media de esta tabla caracteriza únicamente este subconjunto agrupado y no debe confundirse con la estadística global calculada posteriormente por Praeco Análisis sobre los 63 registros de la validación final.

Los resultados muestran que la etapa con mayor coste temporal es claramente la captura y selección de la imagen, con un valor medio global de aproximadamente 3994.2 ms. El reconocimiento mediante Google ML Kit requiere, en promedio, 247.0 ms, mientras que la corrección desarrollada específicamente para Praeco presenta un tiempo medio considerablemente inferior, de 65.8 ms. La detección de términos asociados a alérgenos supone un coste reducido, con un promedio de únicamente 7.6 ms.

Table 4.4: Tiempos medios de procesamiento por producto durante la fase de validación.

Producto	**Captura (ms)**	**OCR (ms)**	**Corrección (ms)**	**Alérgenos (ms)**	**Total (ms)**
Fairy	4060.3	226.3	71.3	6.3	4364.3
KH-7	3944.8	199.0	48.5	5.5	4197.8
Gallina Blanca	3984.0	215.7	59.7	6.0	4265.3
Eroski	4044.7	271.3	121.3	17.7	4455.0
Azucarera	4229.3	333.7	81.3	9.0	4653.3
Alipende	3873.0	246.7	22.3	3.7	4145.7
Colón	3941.0	260.3	63.3	10.3	4275.0
Doussy	3950.7	282.0	55.7	4.3	4292.7
Sanytol	3936.7	203.7	74.7	6.3	4221.3
Media global	**3994.2**	**247.0**	**65.8**	**7.6**	**4314.6**

Considerando conjuntamente las cuatro etapas registradas en este subconjunto por productos, el tiempo medio calculado es aproximadamente 4314.6 ms, es decir, unos 4.31 s. Este valor se obtiene como suma de los tiempos parciales y no corresponde a una medida independiente del tiempo total de ejecución. La estadística global de los 63 registros finales, presentada más adelante, proporciona un tiempo medio total de aproximadamente 4432 ms; la diferencia se debe a que ambos valores se calculan sobre conjuntos de agregación distintos.

Las diferencias observadas entre productos se relacionan, entre otros factores, con la cantidad y complejidad del texto procesado. Por ejemplo, Eroski presenta un tiempo medio de corrección de 121.3 ms, superior a la media global, mientras que para Alipende se reduce a 22.3 ms. De forma similar, el tiempo empleado por el OCR varía entre productos, alcanzando 333.7 ms en Azucarera frente a 199.0 ms en KH-7.

Estos resultados indican que las etapas específicas añadidas por Praeco después del reconocimiento OCR introducen un coste computacional relativamente reducido. En particular, la corrección y la detección de alérgenos representan una fracción pequeña del tiempo global, que se encuentra dominado por el proceso de adquisición y selección de la imagen.

4.11. Papel de Praeco Análisis

La evolución de la metodología experimental motivó el desarrollo de *Praeco Análisis* como una tercera aplicación independiente. Su función consiste en procesar el fichero `experimentos.csv` generado por Praeco Desarrollo y obtener estadísticas sobre las métricas registradas.

Praeco Análisis permite comparar OCR original y texto corregido, identificar casos de

mejora, empeoramiento o ausencia de cambios, estudiar los tiempos de procesamiento y analizar las variables de calidad de imagen. El campo `modo_aprendizaje` permite además separar los experimentos realizados con aprendizaje activo de los efectuados con recursos congelados, evitando mezclar dos condiciones experimentales diferentes en una misma estadística global.

Esta separación mantiene también la arquitectura funcional del proyecto: Praeco Desarrollo se utiliza para experimentar y aprender, Praeco Usuario ejecuta el flujo operativo con recursos validados y Praeco Análisis realiza el tratamiento estadístico de los resultados.

Como puede observarse en la Figura 4.2, Praeco Análisis automatiza el tratamiento de los datos obtenidos durante la validación. En el ejemplo mostrado se han procesado 63 registros con 21 variables. Todos los experimentos corresponden a la fase de validación, con el aprendizaje desactivado, por lo que los recursos lingüísticos permanecieron congelados durante los ensayos.

Debe señalarse que el fichero `experimentos.csv` se ha construido de forma incremental a lo largo del desarrollo de Praeco. Los distintos registros corresponden a sucesivas fases de análisis, ajuste y mejora de la aplicación, durante las cuales se fueron incorporando modificaciones en el procesamiento y en el sistema de corrección. Paralelamente, durante las fases en las que el aprendizaje se encontraba habilitado, se fue enriqueciendo el fichero `diccionario_adaptativo.json` con vocabulario validado a partir de los experimentos realizados. Este recurso constituye, por tanto, conocimiento adquirido durante el desarrollo y posteriormente reutilizado por el corrector. Los resultados obtenidos muestran que la incorporación progresiva de este vocabulario contribuye a mejorar el texto `ocr_corregido`, al permitir reconocer como válidos términos aprendidos previamente y evitar correcciones innecesarias. En la fase de validación final, realizada con `modo_aprendizaje = false`, el diccionario adaptativo se mantiene congelado: se utiliza como recurso de corrección, pero no se modifica con los nuevos experimentos.

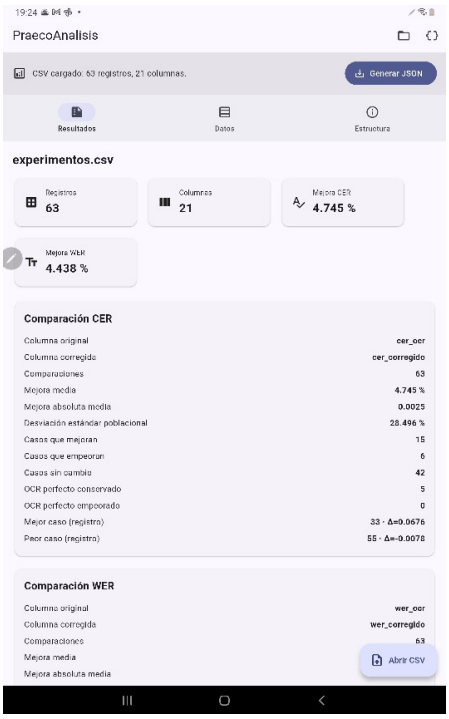
Para el conjunto analizado se obtiene un CER medio de 0.0646 antes de la corrección y de 0.0621 después de aplicar el corrector. De forma análoga, el WER medio disminuye de 0.1834 a 0.1717. Estos resultados muestran que, para este conjunto experimental, la aplicación del corrector produce una reducción global del error de reconocimiento.

Praeco Análisis permite además estudiar el comportamiento del corrector registro a registro, diferenciando los casos que mejoran, empeoran o permanecen sin cambios. Esta información resulta especialmente relevante para evaluar el carácter conservador del sistema y detectar aquellos casos en los que una modificación introducida por el corrector puede resultar contraproducente.

La aplicación proporciona también el análisis del coste temporal. En el conjunto mostrado, el tiempo medio total es de aproximadamente 4432 ms. Por etapas, los valores medios registrados son aproximadamente 4110 ms para la captura, 251 ms para el OCR, 63 ms para la corrección y 8 ms para la detección de alérgenos. Se observa, por tanto, que la mayor

parte del tiempo de ejecución corresponde al proceso de adquisición y selección de la imagen, mientras que la corrección léxica y la detección de alérgenos introducen un coste temporal reducido.

Durante el procesamiento del conjunto experimental, la aplicación permite analizar las métricas CER y WER, estudiar el comportamiento del corrector, obtener estadísticos descriptivos y evaluar el coste temporal de las diferentes etapas del procesamiento.



(a) Resumen general y comparación de las métricas CER y WER.



(b) Evaluación del corrector y análisis del coste temporal.



(c) Análisis avanzado y estadísticos de las variables registradas.



(d) Estadísticos de calidad y tiempos de procesamiento.

Figure 4.2: Pantallas de ejemplo de Praeco Análisis.

4.12. Discusión de los resultados

Los resultados obtenidos a lo largo del desarrollo de Praeco muestran que la evaluación del corrector no puede reducirse únicamente a comprobar si los valores medios de CER o WER disminuyen. Aunque estas métricas permiten cuantificar de forma objetiva la precisión del reconocimiento, resulta igualmente necesario estudiar el comportamiento del corrector registro a registro, su estabilidad y, especialmente, su capacidad para conservar aquellas palabras que el OCR ya ha reconocido correctamente.

Debe tenerse en cuenta, además, que el fichero `experimentos.csv` se ha construido de forma incremental durante el desarrollo del proyecto. Los registros almacenados corresponden a distintas fases de análisis, ajuste y mejora de la aplicación. Paralelamente, durante las etapas en las que el aprendizaje se encontraba habilitado, se fue enriqueciendo el fichero `diccionario_adaptativo.json` con vocabulario validado a partir de los experimentos realizados. Por tanto, el conjunto completo de registros refleja también la evolución del sistema y no debe interpretarse como un único ensayo realizado con una versión estática de Praeco.

Las primeras versiones del corrector mostraron que una estrategia excesivamente permisiva podía solucionar determinados errores del OCR y, simultáneamente, introducir sustituciones incorrectas sobre palabras que originalmente eran válidas. Esta situación explica que algunos experimentos presentasen mejoras locales mientras que las métricas globales permanecían prácticamente constantes o incluso empeoraban. Estos resultados fueron especialmente útiles durante el desarrollo, ya que permitieron identificar la necesidad de aumentar el carácter conservador del corrector.

Como consecuencia de este análisis se incorporaron progresivamente reglas de protección, criterios de confianza, control de conflictos y vocabulario adaptativo. El objetivo dejó de ser corregir el mayor número posible de palabras para pasar a modificar únicamente aquellas para las que existe evidencia suficiente de que se ha producido un error de reconocimiento.

El `diccionario_adaptativo.json` constituye un elemento relevante en esta evolución. Su contenido representa vocabulario validado durante las fases de desarrollo y permite que términos encontrados previamente puedan ser reconocidos posteriormente como palabras válidas. De este modo se evita que el corrector sustituya innecesariamente términos correctos y se facilita la obtención de una salida `ocr_corregido` más precisa. Conviene señalar que este mecanismo no modifica directamente el funcionamiento interno de Google ML Kit ni el texto OCR bruto, sino que actúa en la etapa posterior de corrección desarrollada específicamente en Praeco.

El análisis del conjunto formado por 63 registros, realizado con `modo_aprendizaje = false`, permite evaluar el comportamiento del sistema utilizando los recursos lingüísticos previamente adquiridos pero manteniéndolos congelados durante la validación. En este conjunto, el CER medio pasa de 0.0646 para el OCR original a 0.0621 después de la corrección, mientras que el WER medio disminuye de 0.1834 a 0.1717. Praeco Análisis obtiene una

mejora media del CER del 4.745 % y del WER del 4.438 %.

Sin embargo, esta mejora global no significa que todos los registros mejoren individualmente. En la comparación del CER, 15 registros mejoran, 6 empeoran y 42 permanecen sin cambios. Además, se conservan los cinco casos en los que el OCR era inicialmente perfecto, sin que el corrector introduzca errores en ellos. Estos resultados son coherentes con el criterio conservador adoptado: una proporción elevada de casos sin cambios no debe interpretarse necesariamente como una limitación, puesto que el objetivo es evitar modificaciones cuando no existe suficiente evidencia para realizarlas.

El análisis del WER conduce a una conclusión similar. La reducción de su valor medio indica que el corrector puede aportar una mejora global en el reconocimiento a nivel de palabra, pero siguen existiendo casos particulares en los que la corrección no produce ninguna modificación o puede introducir un empeoramiento. Por ello, el resultado agregado debe analizarse conjuntamente con el número de casos que mejoran, empeoran o permanecen inalterados.

El aprendizaje adaptativo tampoco debe interpretarse como un mecanismo de traducción o normalización automática al castellano. Su finalidad consiste en incorporar conocimiento validado del dominio. Las etiquetas comerciales pueden contener nombres propios, marcas, términos técnicos y vocabulario perteneciente a otros idiomas. Cuando estos términos aparecen realmente en el *ground truth*, deben considerarse palabras válidas y conservarse. Un término extranjero correctamente reconocido no constituye por sí mismo un error de OCR.

Otro aspecto relevante es el coste temporal introducido por las diferentes etapas. Para el conjunto de 63 registros, Praeco Análisis obtiene un tiempo medio total de aproximadamente 4432 ms. La mayor parte corresponde al proceso de captura y selección de la imagen, con aproximadamente 4110 ms de media. El reconocimiento OCR requiere alrededor de 251 ms, mientras que la corrección presenta un tiempo medio próximo a 63 ms y la detección de alérgenos aproximadamente 8 ms. Estos resultados muestran que las etapas específicas de corrección y detección añadidas por Praeco representan una fracción reducida del tiempo total de procesamiento.

Desde el punto de vista funcional, este comportamiento resulta especialmente importante debido al objetivo asistencial de la aplicación. El texto corregido se utiliza posteriormente para la detección de términos asociados a alérgenos y para su reproducción mediante síntesis de voz. Una corrección incorrecta puede, por tanto, propagarse hacia las etapas posteriores. Esta circunstancia justifica que el sistema priorice la conservación de palabras válidas frente a una estrategia más agresiva orientada únicamente a maximizar el número de sustituciones.

Finalmente, la separación del sistema en Praeco Desarrollo, Praeco Usuario y Praeco Análisis ha permitido diferenciar claramente tres funciones. Praeco Desarrollo proporciona los mecanismos de experimentación, aprendizaje y generación de resultados; Praeco Usuario utiliza los recursos lingüísticos obtenidos sin necesidad de *ground truth* ni aprendizaje durante

el uso normal; y Praeco Análisis permite procesar los resultados experimentales, calcular estadísticos, comparar CER y WER, analizar el coste temporal y estudiar el comportamiento del corrector.

4.13. Ensayos exploratorios de captura rápida e iluminación asistida

Después de la validación principal de 63 registros se desarrolló una versión de captura optimizada con una sola fotografía, control de linterna y mejora condicional. Los resultados de esta etapa constituyen **pruebas exploratorias posteriores**, no una ampliación de la muestra de validación del corrector ni un ensayo controlado de CLAHE.

En dos registros de la prueba con umbral de linterna 150 se obtuvieron los valores de la Tabla 4.5. Las imágenes fueron aceptadas y el OCR corregido alcanzó CER y WER nulos en ambos casos.

Table 4.5: Dos ensayos exploratorios de captura rápida con umbral de iluminación 150.

Indicador	**Mejillones Galica**	**Don Simón**
Brillo medio	132,93	163,73
Nitidez	511,41	402,35
Contraste	44,46	37,52
Tiempo de captura (ms)	1842	1530
Tiempo OCR (ms)	194	114
CER antes de corrección	0,0172	0,0167
CER después de corrección	0	0
WER después de corrección	0	0

El tiempo medio de captura de estas dos observaciones fue 1686 ms, frente a los aproximadamente 4110 ms de captura media de la validación anterior. La diferencia descriptiva es de 2424 ms (aproximadamente un 59 %), pero **no puede interpretarse como una mejora causal demostrada**, porque los conjuntos de imágenes y las condiciones experimentales no son equivalentes y el alcance de los temporizadores debe comprobarse. El brillo de 132,93 corresponde a la imagen registrada y no prueba por sí solo el estado de la linterna durante la vista previa.

En ambos registros el OCR se realizó sobre la imagen original; por ello, **no se atribuye a CLAHE la reducción de CER o WER**. Para demostrar la aportación de esta técnica será necesario comparar, sobre las mismas capturas, OCR original frente a OCR con CLAHE, con el corrector y los recursos de aprendizaje congelados. También se requiere registrar el brillo previo, el estado de la linterna y el tiempo de estabilización para caracterizar el control de iluminación.

4.14. Conclusiones del análisis experimental

Los resultados obtenidos durante las diferentes etapas de desarrollo y validación permiten extraer las siguientes conclusiones:

- Las métricas CER y WER permiten cuantificar objetivamente tanto los errores producidos por el OCR como el efecto posterior del corrector desarrollado para Praeco.
- Las primeras versiones demostraron que una estrategia de corrección excesivamente permisiva puede introducir falsos positivos y empeorar palabras que originalmente habían sido reconocidas correctamente.
- La incorporación de reglas de protección, criterios de confianza y mecanismos de validación ha permitido evolucionar hacia una estrategia de corrección más conservadora.
- El aprendizaje adaptativo permite construir progresivamente un vocabulario específico a partir de información validada durante la fase de desarrollo. Este conocimiento queda almacenado principalmente en `diccionario_adaptativo.json` y puede ser reutilizado posteriormente por el corrector.
- El fichero `experimentos.csv` refleja la evolución experimental de Praeco, puesto que se ha ido ampliando durante las diferentes etapas de análisis y mejora de la aplicación. Por esta razón, sus registros no deben interpretarse necesariamente como resultados obtenidos con una única versión estática del sistema.
- Los experimentos realizados con `modo_aprendizaje = true` pertenecen a la fase de adquisición de conocimiento y no deben confundirse con una validación independiente del sistema.
- La utilización de `modo_aprendizaje = false` permite congelar los recursos adquiridos y evaluar el comportamiento del corrector sin que los propios datos de validación modifiquen simultáneamente el sistema.
- En el conjunto analizado de 63 registros con aprendizaje desactivado se obtiene una mejora global de las métricas. El CER medio disminuye de 0.0646 a 0.0621 y el WER medio de 0.1834 a 0.1717, correspondientes a mejoras medias calculadas por Praeco Análisis del 4.745 % y del 4.438 %, respectivamente.
- La mejora global no implica una mejora individual en todos los casos. Para CER, 15 registros mejoran, 6 empeoran y 42 permanecen sin cambios. Los cinco casos con OCR inicialmente perfecto se conservan sin introducir errores, lo que constituye un indicador relevante del carácter conservador del corrector.

- La auditoría manual del *ground truth* resulta imprescindible para que los valores obtenidos de CER y WER representen correctamente el error real del sistema.
- El vocabulario multilingüe, los nombres propios, las marcas y otros términos presentes legítimamente en las etiquetas deben conservarse y no considerarse automáticamente errores. El aprendizaje adaptativo no constituye un mecanismo de traducción.
- El análisis temporal muestra que el coste añadido específicamente por la corrección y la detección de alérgenos es reducido respecto al tiempo total de procesamiento. La mayor parte del tiempo corresponde al proceso de adquisición y selección de la imagen.
- La separación entre Praeco Desarrollo, Praeco Usuario y Praeco Análisis permite distinguir claramente las fases de experimentación y aprendizaje, utilización del asistente y evaluación de los resultados.

En conjunto, los resultados permiten concluir que el corrector de Praeco produce una mejora global moderada de las métricas CER y WER en el conjunto de validación analizado, aunque dicha mejora no se produce de manera uniforme en todos los registros. Este comportamiento confirma la conveniencia de utilizar una estrategia conservadora, en la que una palabra únicamente se modifica cuando existe evidencia suficiente para realizar la corrección.

La evolución del `diccionario_adaptativo.json` permite incorporar conocimiento validado durante el desarrollo y reutilizarlo posteriormente sin modificar los recursos durante la fase de validación. La combinación de aprendizaje previo, congelación de los recursos y evaluación posterior proporciona una metodología más rigurosa que la utilización simultánea de aprendizaje y validación sobre los mismos datos.

Por último, el desarrollo de Praeco no se limita al reconocimiento óptico de caracteres, sino que integra adquisición y selección automática de imágenes, OCR, corrección léxica, aprendizaje adaptativo, detección de términos asociados a alérgenos, síntesis de voz y herramientas específicas para la evaluación experimental. Los resultados obtenidos muestran que esta arquitectura puede ejecutarse en un dispositivo móvil con un coste adicional reducido para las etapas de corrección y detección, manteniendo como criterio fundamental la conservación del texto correctamente reconocido.

Chapter 5

Conclusiones y trabajos futuros

5.1. Conclusiones

El desarrollo realizado en este Trabajo Fin de Máster ha permitido diseñar, implementar y evaluar un asistente móvil orientado a facilitar el acceso a la información textual presente en etiquetas de productos a personas con discapacidad visual. El sistema desarrollado, denominado Praeco, combina técnicas de visión artificial, reconocimiento óptico de caracteres, procesamiento lingüístico, aprendizaje adaptativo, detección de términos asociados a alérgenos y síntesis de voz dentro de una aplicación ejecutable sobre dispositivos Android convencionales.

El trabajo ha evolucionado desde un prototipo centrado fundamentalmente en la captura y reconocimiento OCR hasta un ecosistema formado por tres aplicaciones diferenciadas: *Praeco Desarrollo*, *Praeco Usuario* y *Praeco Análisis*. Esta separación permite distinguir claramente las tareas de experimentación y aprendizaje, el funcionamiento normal del asistente y el análisis posterior de los resultados.

Praeco Desarrollo incorpora la instrumentación necesaria para evaluar el sistema mediante *ground truth*, calcular las métricas CER y WER, registrar los tiempos de ejecución, almacenar las variables asociadas a la calidad de imagen y realizar aprendizaje adaptativo. Praeco Usuario utiliza los recursos obtenidos durante el desarrollo, pero elimina los mecanismos de evaluación y aprendizaje que no son necesarios durante el funcionamiento normal. Finalmente, Praeco Análisis permite procesar los ficheros experimentales y obtener automáticamente estadísticas sobre las métricas de reconocimiento, comportamiento del corrector, calidad de imagen y coste temporal.

Uno de los resultados más relevantes del trabajo ha sido comprobar que la corrección posterior al OCR debe realizarse de forma conservadora. Las primeras versiones mostraron que una estrategia excesivamente permisiva podía corregir determinados errores y, al mismo tiempo, modificar palabras que originalmente habían sido reconocidas correctamente. Como consecuencia, el corrector evolucionó incorporando reglas de protección, distancia de Leven-

shtein, criterios fonéticos, umbrales dependientes de la longitud de las palabras, criterios de confianza y vocabulario adaptativo.

El aprendizaje adaptativo permite incorporar progresivamente vocabulario validado durante la fase de desarrollo. Este conocimiento se almacena principalmente

en `diccionario_adaptativo.json` y puede reutilizarse posteriormente para reconocer como válidos términos previamente aprendidos. Este mecanismo no modifica el funcionamiento interno de Google ML Kit ni el OCR bruto, sino que mejora la etapa posterior de corrección y contribuye a obtener un texto `ocr_corregido` más preciso.

Debe señalarse que el fichero `experimentos.csv` se ha construido progresivamente durante las distintas etapas de desarrollo, análisis y mejora de Praeco. Por este motivo, el conjunto completo de registros refleja también la evolución del sistema y de los recursos lingüísticos empleados. Durante las fases con aprendizaje activo se fue enriqueciendo el `diccionario_adaptativo.json`, mientras que en la validación final los recursos fueron congelados mediante `modo_aprendizaje = false`, evitando que los propios datos utilizados para validar el sistema modificasen simultáneamente el conocimiento adquirido.

Los resultados obtenidos en el conjunto de 63 registros analizado con los recursos congelados muestran una mejora global moderada después de aplicar el corrector. El CER medio disminuye de 0.0646 a 0.0621 y el WER medio de 0.1834 a 0.1717. Praeco Análisis calcula una mejora media del CER del 4.745 % y del WER del 4.438 %.

Esta mejora global no se produce de forma uniforme en todos los experimentos. En el análisis del CER, 15 registros mejoran, 6 empeoran y 42 permanecen sin cambios. Asimismo, los cinco registros en los que el OCR inicial era perfecto se conservan sin introducir errores adicionales. Este comportamiento resulta coherente con la estrategia conservadora adoptada: cuando el sistema no dispone de evidencia suficiente para realizar una sustitución, resulta preferible conservar el término reconocido originalmente.

La auditoría del *ground truth* ha resultado igualmente fundamental para la evaluación experimental. CER y WER únicamente proporcionan resultados fiables cuando el texto utilizado como referencia representa fielmente el contenido de la etiqueta. Este proceso permitió además comprobar la existencia legítima de nombres comerciales, nombres propios y términos pertenecientes a diferentes idiomas. Por tanto, el aprendizaje adaptativo no debe interpretarse como un mecanismo de traducción al castellano: una palabra extranjera presente realmente en la etiqueta constituye un término válido y debe conservarse.

Otro resultado relevante corresponde al análisis temporal. Para los 63 registros evaluados se obtiene un tiempo medio total de aproximadamente 4432 ms. La mayor parte corresponde a la adquisición y selección de la imagen, con aproximadamente 4110 ms, mientras que el reconocimiento OCR requiere alrededor de 251 ms. La corrección presenta un coste medio próximo a 63 ms y la detección de términos asociados a alérgenos aproximadamente 8 ms. Por tanto, las etapas específicas añadidas por Praeco después del OCR representan una fracción reducida del tiempo global de procesamiento.

Como evolución posterior a la validación principal, se ha incorporado una estrategia de captura única con evaluación previa, iluminación auxiliar continua y preprocesamiento condicional mediante CLAHE. Dos ensayos exploratorios muestran tiempos de captura de 1842 y 1530 ms, pero no constituyen una validación controlada de la mejora ni permiten atribuir a CLAHE un incremento de precisión.

En conjunto, los resultados obtenidos permiten concluir que Praeco constituye un prototipo funcional capaz de integrar en un dispositivo móvil las principales etapas necesarias para capturar una etiqueta, seleccionar una imagen adecuada, reconocer su contenido textual, aplicar una corrección conservadora, identificar determinados términos asociados a alérgenos y comunicar el resultado mediante síntesis de voz.

5.2. Aportaciones principales del trabajo

Las principales aportaciones realizadas durante el desarrollo del proyecto pueden resumirse en los siguientes puntos:

- Desarrollo de una aplicación móvil accesible basada en Flutter y orientada a personas con discapacidad visual.
- Implementación inicial de adquisición y selección entre tres fotografías, posteriormente optimizada mediante evaluación de fotogramas en vista previa y captura automática única, con iluminación auxiliar estabilizada.
- Incorporación de preprocesamiento condicional de imagen mediante CLAHE y ajustes de iluminación, con conservación y comparación visual de la fotografía original y la procesada.
- Definición de una región de interés (*Region of Interest*, ROI) para reducir la información innecesaria y concentrar el reconocimiento sobre la zona relevante de la etiqueta.
- Integración de Google ML Kit como motor de reconocimiento óptico de caracteres.
- Desarrollo de un sistema propio de filtrado y reconstrucción del texto obtenido mediante OCR.
- Implementación de un corrector léxico híbrido basado en distancia de Levenshtein, información fonética, diferencias de longitud, criterios de confianza y mecanismos de protección de términos válidos.
- Desarrollo de un mecanismo de aprendizaje incremental basado en los errores observados durante los experimentos y en la incorporación de vocabulario validado al `diccionario_adaptativo.json`.
- Implementación de mecanismos específicos para proteger números, unidades, palabras cortas, nombres propios, marcas y vocabulario multilingüe presente legítimamente en las etiquetas.
- Detección automática de términos asociados a diferentes categorías de alérgenos y

consideración de expresiones de ausencia o negación, como “sin gluten”, para reducir falsos positivos.
- Integración de síntesis de voz para comunicar al usuario el texto reconocido y las advertencias obtenidas.

5.3. Trabajos futuros

Los resultados obtenidos permiten identificar varias líneas de continuación que pueden mejorar tanto la robustez técnica de Praeco como la validez externa de su evaluación.

- **Ampliación del corpus experimental.** Incrementar el número y la diversidad de etiquetas, condiciones de iluminación, tipografías, materiales de envase y dispositivos permitiría caracterizar con mayor precisión el comportamiento del sistema y estudiar su capacidad de generalización.
- **Evaluación con usuarios.** La validación realizada en este TFM se centra principalmente en métricas técnicas. Un trabajo posterior debería incorporar pruebas de usabilidad y accesibilidad con personas con discapacidad visual, evaluando aspectos como facilidad de captura, comprensión de los mensajes, tiempos percibidos y utilidad en situaciones reales.
- **Mejora del tratamiento multilingüe.** Las etiquetas comerciales pueden contener nombres propios y términos en distintos idiomas. Resulta de interés ampliar los mecanismos de protección e identificación lingüística para reducir correcciones indebidas sin convertir el corrector en un sistema de traducción.
- **Evaluación de mecanismos contextuales.** Los recursos basados en n-gramas se encuentran disponibles en el proyecto, pero la corrección contextual no forma parte de la ruta activa evaluada en los 63 registros finales. Una línea futura consiste en activar y validar experimentalmente este mecanismo mediante un estudio de ablación que permita cuantificar su aportación independiente.
- **Ampliación de la detección de alérgenos.** El módulo actual identifica términos asociados a determinadas categorías y expresiones de negación. Su evolución debería contemplar vocabulario más amplio, variantes lingüísticas y una validación específica sobre etiquetado alimentario. En cualquier caso, esta función debe mantenerse como ayuda informativa y no como certificación de seguridad alimentaria.
- **Optimización de la adquisición.** Dado que la captura y selección de la imagen concentra la mayor parte del tiempo total de procesamiento, la versión optimizada debe validarse con un corpus ampliado y condiciones controladas, comparando el tiempo extremo a extremo y la robustez de la activación de la linterna.
- **Evaluación específica de CLAHE.** Realizar un ensayo de ablación con las mismas imágenes, sin mejora, con CLAHE y con ajustes de brillo; medir CER, WER, latencia y posibles empeoramientos, manteniendo congelado el corrector.

- **Comparación entre dispositivos y motores OCR.** La evaluación en diferentes terminales Android y la comparación controlada con otros motores de reconocimiento permitirían determinar qué parte del rendimiento depende de la plataforma y qué parte corresponde específicamente a las etapas desarrolladas en Praeco.

Estas líneas permitirían evolucionar Praeco desde el prototipo funcional y experimental desarrollado en este TFM hacia una herramienta asistencial más robusta, manteniendo como principio metodológico la separación entre aprendizaje, validación y uso final.

Bibliography

M. Hersh and M. Johnson. *Assistive Technology for Visually Impaired and Blind People*. Springer, 2008.

R. C. Gonzalez and R. E. Woods. *Digital Image Processing*. Pearson Education, 4ª edition, 2018.

R. Szeliski. *Computer Vision: Algorithms and Applications*. Springer, 2011.

R. Smith. An overview of the tesseract ocr engine. In *Proceedings of ICDAR*, 2007.

V. I. Levenshtein. Binary codes capable of correcting deletions, insertions and reversals. *Soviet Physics Doklady*, 1966.

D. Jurafsky and J. Martin. *Speech and Language Processing*. Prentice Hall, 3ª edition, 2023.

Microsoft. Seeing ai. `https://www.microsoft.com/en-us/ai/seeing-ai`, 2026. Consultado en mayo de 2026.

Google. Google lens. `https://lens.google`, 2026a. Consultado en mayo de 2026.

Google. Android developers documentation. `https://developer.android.com`, 2026b. Consultado en mayo de 2026.

Google. Flutter documentation. `https://flutter.dev`, 2026c. Consultado en mayo de 2026.

Google. Dart programming language. `https://dart.dev`, 2026d. Consultado en mayo de 2026.

Google. Google ml kit documentation. `https://developers.google.com/ml-kit`, 2026e. Consultado en mayo de 2026.

Flutter Community. Flutter tts package. `https://pub.dev/packages/flutter_tts`, 2026a. Consultado en mayo de 2026.

Flutter Community. Camera package for flutter. `https://pub.dev/packages/camera`, 2026b. Consultado en mayo de 2026.

S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4ª edition, 2021.

C. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

W3C. Web content accessibility guidelines (wcag). `https://www.w3.org/WAI/standards-guidelines/wcag/`, 2026. Consultado en mayo de 2026.

OpenAI. Openai api documentation. `https://platform.openai.com/docs`, 2026. Consultado en mayo de 2026.

G. Bradski. The opencv library. *Dr. Dobb's Journal of Software Tools*, 2000.

F. Loitsch. Asynchronous programming in dart. Google Developers Documentation, 2024.

Appendix A

Software desarrollado

A.1. Introducción

Como parte de este Trabajo Fin de Máster se ha desarrollado *Praeco*, un sistema software orientado a facilitar el acceso a la información textual presente en el etiquetado de productos mediante técnicas de visión artificial, reconocimiento óptico de caracteres y síntesis de voz. La versión final se organiza en tres aplicaciones independientes: **Praeco Desarrollo**, **Praeco Usuario** y **Praeco Análisis**.

A.2. Repositorio y versión de referencia

El código fuente correspondiente a la versión final de Praeco se encuentra disponible públicamente en el repositorio GitHub del proyecto:

`https://github.com/fjlopezlp-beep/Asistente-Discapacidad-Visual`

La versión utilizada como referencia para la entrega final se identifica mediante la etiqueta `v2.3.1-TFM` y la *release* **Praeco v2.3.1 – TFM**. Las tres aplicaciones se identifican con la versión `2.3.1+1`.

A.3. Estructura del repositorio

El repositorio separa las tres aplicaciones en los directorios `Praeco_Desarrollo`, `Praeco_Usuario` y `Praeco_Analisis`. Cada aplicación conserva su propio código fuente, recursos, configuración Android y ficheros `pubspec.yaml` y `pubspec.lock`. No se incluyen cachés de Flutter, resultados de compilación ni configuraciones locales del entorno de desarrollo.

A.4. Praeco Desarrollo

Praeco Desarrollo corresponde a la aplicación utilizada durante las fases de implementación, aprendizaje, experimentación y validación. Integra la adquisición y evaluación de calidad de la imagen, selección de la región de interés, OCR mediante Google ML Kit, filtrado y reconstrucción del texto, corrección léxica, recursos de aprendizaje adaptativo, detección de términos asociados a alérgenos, síntesis de voz y evaluación mediante CER y WER. Durante la validación final, los recursos de aprendizaje se mantienen congelados.

A.5. Praeco Usuario

Praeco Usuario constituye la versión destinada al funcionamiento normal del asistente. No requiere la introducción de *ground truth* ni realiza el cálculo de CER y WER durante su utilización. Emplea los recursos lingüísticos previamente obtenidos y congelados, manteniendo estable el comportamiento del corrector.

A.6. Praeco Análisis

Praeco Análisis es una aplicación independiente destinada al tratamiento estadístico de los resultados experimentales. Permite estudiar CER, WER, comportamiento del corrector, variables de calidad de imagen y tiempos de procesamiento. No necesita los diccionarios empleados por el corrector, puesto que analiza resultados previamente obtenidos y no ejecuta de nuevo el proceso de corrección.

A.7. Requisitos de compilación

Cada aplicación dispone de su propio fichero `pubspec.yaml`. Una vez instalado y configurado Flutter, las dependencias pueden recuperarse mediante `flutter pub get`; la aplicación puede ejecutarse con `flutter run` y puede generarse un APK de distribución mediante `flutter build apk -release`.

A.8. Reproducibilidad

La combinación del código fuente, los recursos necesarios, la definición y bloqueo de dependencias, la etiqueta Git específica y la *release* asociada permite identificar de forma inequívoca el estado del software utilizado como referencia para el Trabajo Fin de Máster.

A.9. Datos generados durante la experimentación

Durante la ejecución de Praeco Desarrollo se utiliza el directorio `TFM_OCR` del dispositivo Android para almacenar datos generados durante los experimentos. Este directorio constituye un espacio de trabajo y no forma parte del código fuente necesario para compilar la aplicación.

Para garantizar la trazabilidad de la evaluación presentada en esta memoria se ha preparado un conjunto de validación independiente, publicado como archivo adicional de la *release* `v2.3.1-TFM`. El archivo `Praeco_Dataset_Validacion_TFM.zip` contiene el fichero de resultados correspondiente a 63 registros de validación con el aprendizaje desactivado, las imágenes asociadas y la información de trazabilidad disponible.

A.10. Disponibilidad del software

La versión de referencia utilizada para la entrega de este Trabajo Fin de Máster es **Praeco v2.3.1 – TFM**, identificada mediante la etiqueta `v2.3.1-TFM` en el repositorio indicado anteriormente.